DSIC
DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y COMPUTACIÓN

Escuela Técnica Superior de Ingeniería Informática etsinf

# Gestión de Datos

## Grado en Ciencias de Datos

### Proyecto

### Traslado del DSIC



UNIVERSITAT POLITÈCNICA DE VALÈNCIA

# Índice

**PARTE 1: CASSANDRA** ..... 3
1.1 Diagrama de clases orientado a la agregación ..... 3
1.1.1 Descripción final del SI: Traslado del DSIC ..... 3
1.1.2 Diagrama de clases final ..... 4
1.1.3 Diagrama de clases orientado a la agregación ..... 5
1.2 Tarea 1: Diseño lógico de la base de datos Cassandra ..... 6
1.2.1 Workflow del SI ..... 6
1.2.2 Transformación diagrama de clases a tablas Cassandra ..... 7
1.2.3 Transformación de los accesos ..... 11
1.2.4 Diagrama Chebotko ..... 17
1.3 Tarea 2: Creación y carga de la base de datos Cassandra ..... 20
1.3.1 Diagrama de la base de datos Oracle generado con el Datamodeler ..... 20
1.3.2 Esquema de la BD en Cassandra ..... 20
1.3.3 Carga de la BD Cassandra ..... 22
1.3.4 Consultas sobre Cassandra ..... 27
**2 PARTE 2: NEO4J** ..... 30
2.1 Tarea 3: Diseño lógico ..... 30
2.1.1 Descripción final del SI: Compañía de Alquiler de Vehículos ..... 30
2.1.2 Diagrama de clases final ..... 31
2.1.3 Esquema de Neo4j ..... 32
2.2 Tarea 4: Creación y carga de la base de datos Neo4j ..... 35
2.2.1 Diagrama de la base de datos Oracle generado con el Datamodeler ..... 35
2.2.2 Creación de la base de datos Traslado DSIC ..... 36
2.2.3 Carga de los nodos ..... 36
2.2.4 Carga de las relaciones entre nodos ..... 38
2.2.5 Definición de las restricciones ..... 39
2.2.6 Esquema Neo4j ..... 40
2.3 Tarea 5: Consultas en Cypher ..... 41

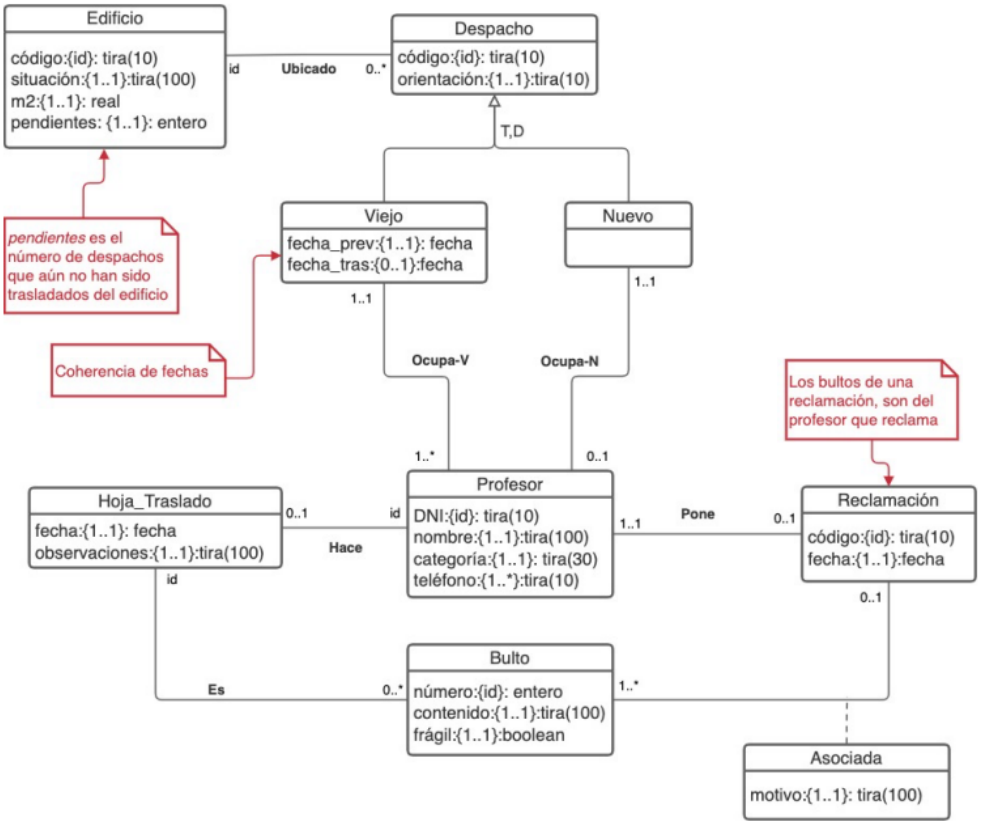
# PARTE 1: CASSANDRA

## 1.1 Diagrama de clases orientado a la agregación

### 1.1.1 Descripción final del SI: Traslado del DSIC

El DSIC con el fin de optimizar el cambio a su nueva sede departamental ha decidido diseñar una base de datos para facilitar la mudanza. El departamento dispone de despachos que se identifican por un código que es único dentro de cada edificio de los que dispone. Los edificios del departamento se identifican por un código único dentro de la UPV y se conoce su situación y su superficie en metros cuadrados. De los despachos interesa saber la orientación (N, S, E, W) y también cuántos despachos están aún pendientes de traslado. La mudanza se realiza de los despachos viejos a los nuevos. Cada profesor, del cual se conoce su DNI, que lo identifica, su nombre, uno o varios teléfonos, su categoría laboral, y el despacho viejo que ocupaba, tiene asignado un despacho nuevo. Los despachos viejos, generalmente, estaban compartidos por más de un profesor y todos estaban ocupados. Los nuevos, por el contrario, son despachos individuales y hay algunos, que por el momento, pueden quedar libres. Se ha elaborado una planificación en la que se indica el día previsto en que se realizará el traslado de un despacho viejo a uno nuevo. Todos los profesores de un despacho se trasladan el mismo día. Como puede haber retrasos, en el momento en que se realice realmente el traslado se debe almacenar el día exacto del mismo. Antes del traslado, cada profesor debe presentar en la secretaría un documento llamado “hoja de traslado” en el que debe figurar la fecha de entrega de la hoja, el profesor que la presenta y las observaciones que considere oportunas para la empresa de mudanzas. Además, dicha hoja debe incluir una lista de los bultos que se van a trasladar. Cada bulto se identifica por un número que es único dentro de cada hoja y se debe indicar claramente su contenido y si es frágil o no. Es posible que algún bulto resulte dañado en el traslado, si esto ocurre, un profesor puede presentar en la secretaría una reclamación (como mucho) en la que debe especificar cuáles de sus bultos han sufrido daños y de qué tipo (pérdida, rotura, …). A cada reclamación, se le asignará un código único en el momento en que se entrega y se guardará la fecha en que se presentó.

### 1.1.2 Diagrama de clases final




Las restricciones de integridad que no se han podido expresar en :

1) El atributo orientación de la clase Edificio toma un valor entre {N,S,W,E}.

2) El atributo fecha_prev de la subclase Viejo deber ser mayor al atributo fecha_entrega de la clase Hoja_Traslado.

3) El atributo fecha_realizado de la subclase Viejo deber ser mayor al atributo fecha_entrega de la clase Hoja_Traslado.

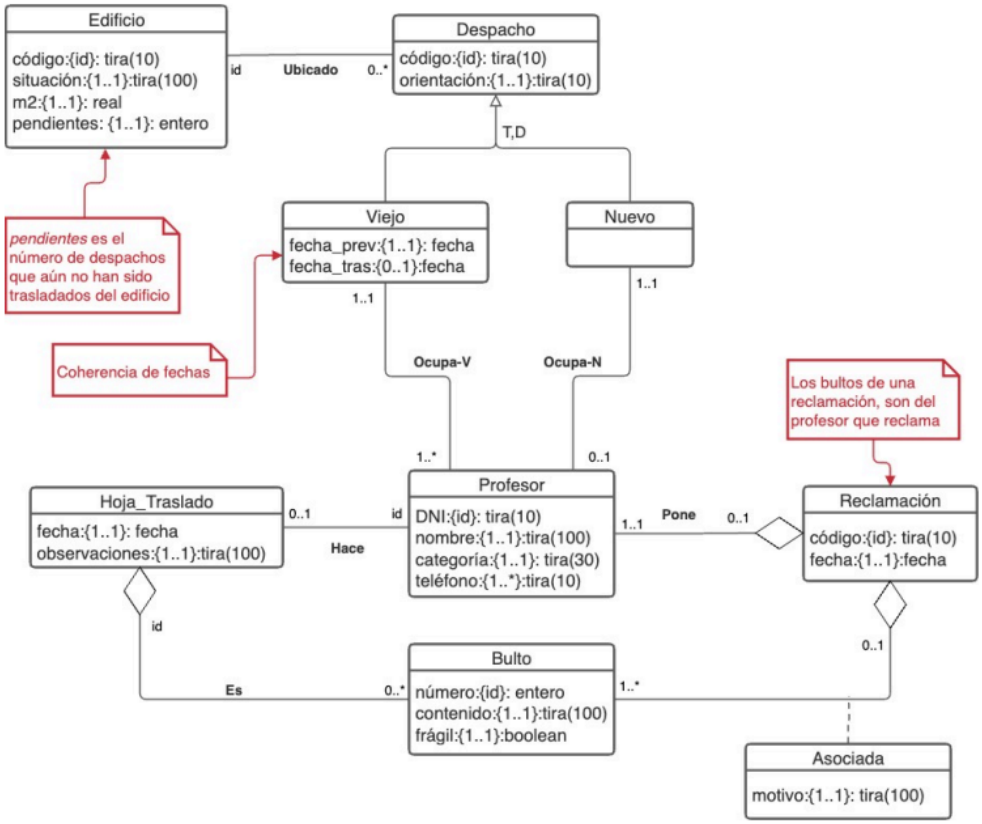
4) El atributo desp_nuevos_pendientes de Edificio es un atributo derivado que se mantendrá automáticamente por el sistema y su valor será en todo momento la cantidad de despachos nuevos que están pendientes de traslado de cada edificio.

5) El atributo desp_viejos_pendientes de Edificio es un atributo derivado que se mantendrá automáticamente por el sistema y su valor será en todo momento la cantidad de despachos viejos que están pendientes de traslado de cada edificio, es decir todos aquellos despachos que el atributo fecha_realizado sea nulo.

6) El atributo fecha_rec de la clase Reclamación debe ser mayor al atributo fecha_realizado de la subclase Viejo.

### 1.1.3 Diagrama de clases orientado a la agregación

Se modifica el diagrama de clases del caso y se orienta a la agregación estructural que pueda existir en la información. El diagrama orientado a la agregación se presenta abajo:

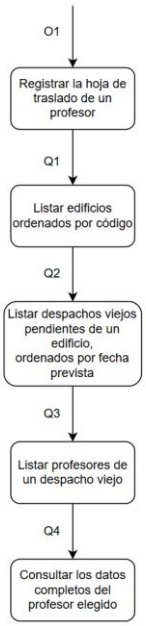



Las restricciones de integridad que no se han podido expresar en son las mismas que se en el diagrama de clases original. Además es necesario tener en cuenta los accesos que se supone que va a tener la información. Estos accesos se representan mediante el Workflow del sistema de información.

## 1.2 Tarea 1: Diseño lógico de la base de datos Cassandra

### 1.2.1 Workflow del SI

El workflow que se incluye a continuación, presenta algunos accesos que se quieren tener en cuenta para el diseño en Cassandra:




Patrones de acceso:

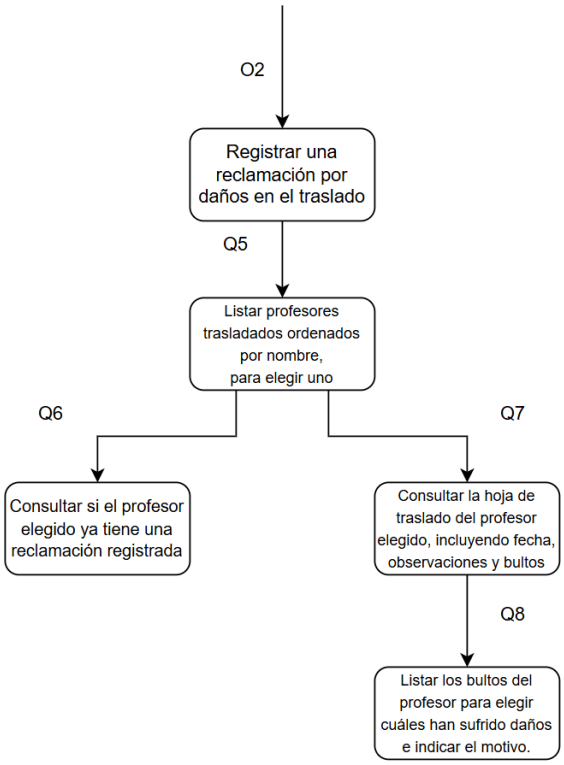
O1: Registrar la hoja de traslado de un profesor.

Q1: Listar edificios ordenados por código.

Q2: Listar despachos viejos pendientes de un edificio, ordenados por fecha prevista.

Q3: Listar profesores de un despacho viejo.

Q4: Consultar los datos completos del profesor elegido.

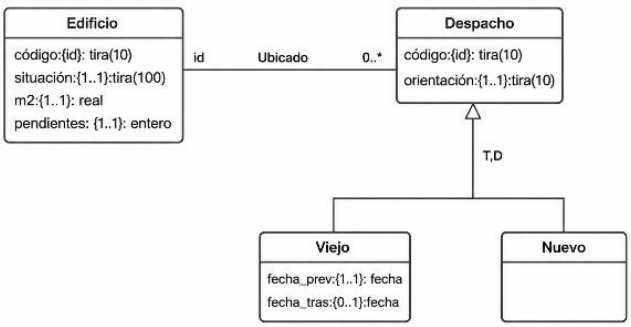



Patrones de acceso:
O2: Registrar una reclamación por daños en el traslado
Q5: Listar profesores trasladados ordenados por nombre, para elegir uno.
Q6: Consultar si el profesor elegido ya tiene una reclamación registrada.
Q7: Consultar la hoja de traslado del profesor elegido, incluyendo fecha, observaciones y bultos
Q8: Listar los bultos del profesor para elegir cuáles han sufrido daños e indicar el motivo.

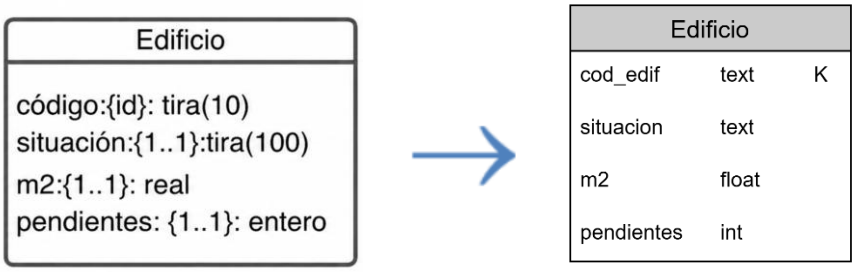
## 1.2.2 Transformación diagrama de clases a tablas Cassandra

### *Transformación de las clases y asociaciones*

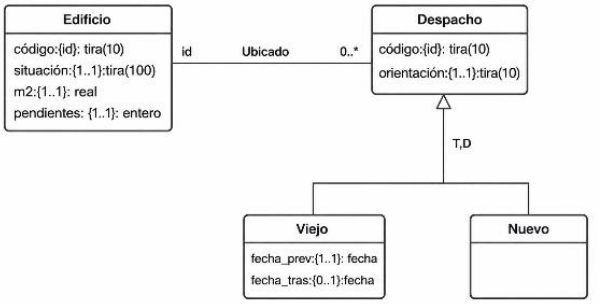
#### Clases *Edificio* y *Despacho*

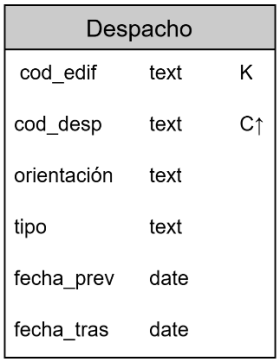



Dado que la cardinalidad mínima de Edificio en Ubicado es 0, es decir que podemos tener información de edificios sin despachos, definimos una tabla para representar esta clase:

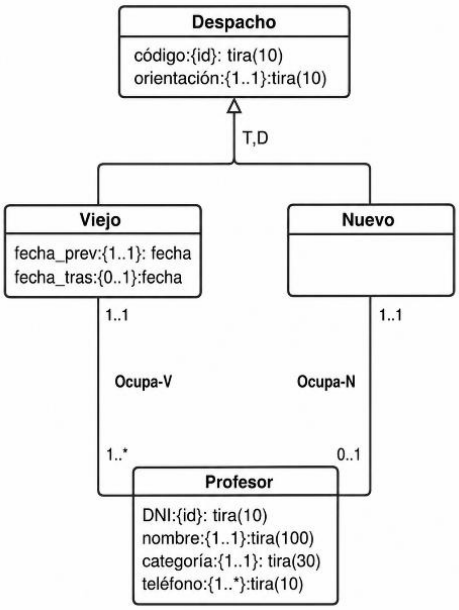



Para representar la asociación Ubicado entre Edificio y Despacho, junto con la especialización total y disjunta de Despacho en Viejo y Nuevo, definimos otra tabla donde se incluye toda la información del despacho. Añadimos el atributo tipo para distinguir si se trata de un despacho viejo o nuevo. Los atributos fecha_prev y fecha_tras solo tendrán valor cuando el despacho sea de tipo viejo.

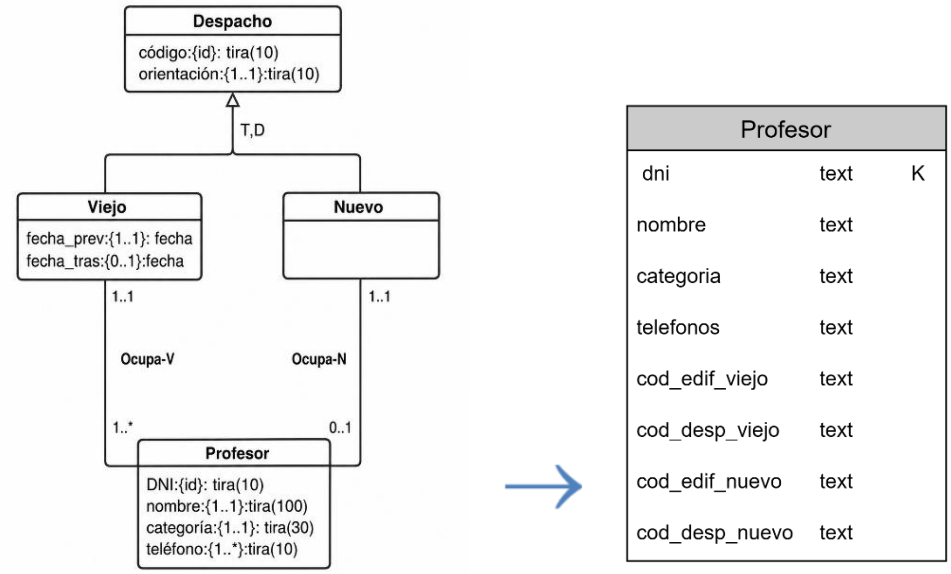


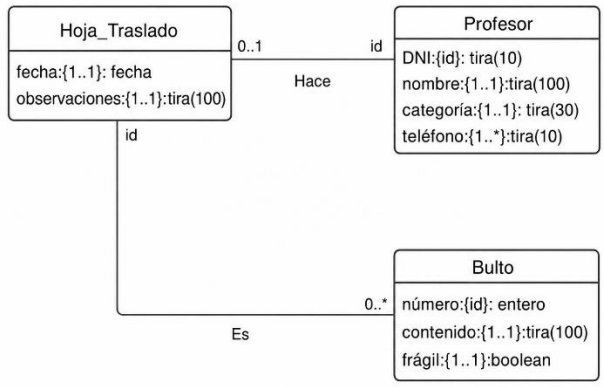
## Clase *Profesor*




Definimos una tabla para representar la clase Profesor. Además, incluimos en esta tabla los códigos del despacho viejo que ocupaba y del despacho nuevo que tiene asignado. Cada profesor ocupa un único despacho viejo, aunque un despacho viejo puede estar compartido por varios profesores. En cambio, cada profesor tiene asignado un despacho nuevo, y los despachos nuevos son individuales. Por tanto, las referencias a ambos despachos se incorporan como atributos en la tabla Profesor.



## Clases *Hoja_Traslado y Bulto*

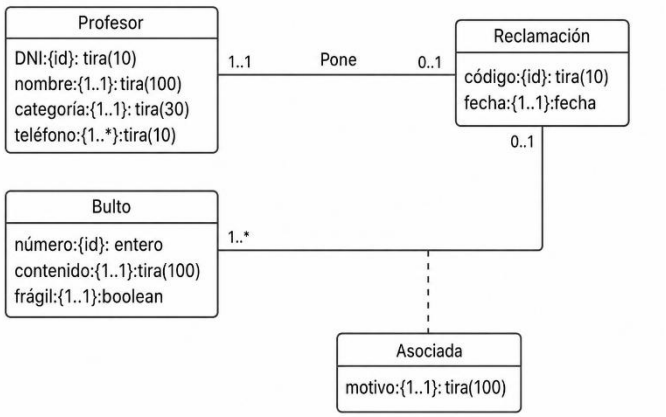



Para representar la asociación Hace entre Profesor y Hoja_Traslado, se incluye el identificador del profesor en la tabla Hoja_traslado. Como cada hoja de traslado pertenece a un único profesor y un profesor puede presentar como mucho una hoja, se toma el dni como clave de la tabla.
Además, para representar la asociación Es entre Hoja_Traslado y Bulto, se define el tipo de usuario Bulto. Como los bultos no tienen existencia independiente fuera de la hoja, se almacenan como una colección dentro de la tabla Hoja_traslado.

| Hoja_traslado | | |
|---|---|---|
| dni | text | K |
| fecha_hoja | date | |
| observaciones | text | |
| bultos | {Bulto} | |

| UDT Bulto | |
|---|---|
| numero | int |
| contenido | text |
| fragil | bool |

## Clases *Reclamación y Bulto_reclamado*




Para representar la asociación Pone entre Profesor y Reclamación, se incluye el identificador del profesor en la tabla Reclamación. Cada reclamación se identifica mediante un código único y almacena la fecha en la que fue presentada.
Además, una reclamación debe indicar qué bultos han sufrido daños y cuál ha sido el motivo de cada daño. Para ello, se define el tipo de usuario Bulto_reclamado, que almacena los datos del bulto afectado junto

con el motivo de la reclamación. La tabla Reclamación almacena estos bultos reclamados como una colección.

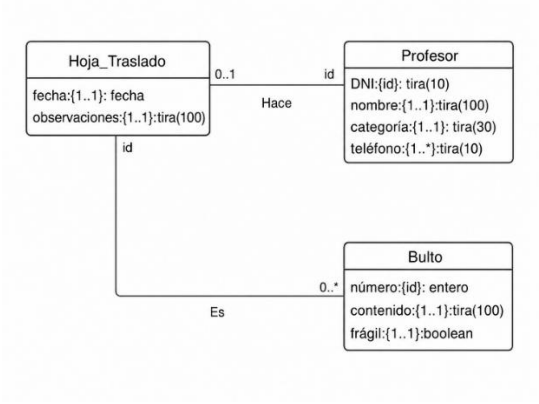
| Reclamacion | | |
|---|---|---|
| cod_rec | text | K |
| dni | text | |
| fecha_rec | date | |
| bultos_rec | {Bulto_reclamado} | |

| UDT Bulto_reclamado | |
|---|---|
| numero | int |
| motivo | text |

**UDT Bulto_reclamado se ha cambiado para que funcione bien la carga de datos

## 1.2.3 Transformación de los accesos

### O1: Registrar la hoja de traslado de un profesor




Como se ha decidido representar la asociación Hace entre Profesor y Hoja_Traslado, y la asociación Es entre Hoja_Traslado y Bulto, mediante la tabla Hoja_traslado, ya tenemos creada la tabla asociada a este acceso.

| Hoja_traslado | | |
|---|---|---|
| dni | text | K |
| fecha_hoja | date | |
| observaciones | text | |
| bultos | {Bulto} | |

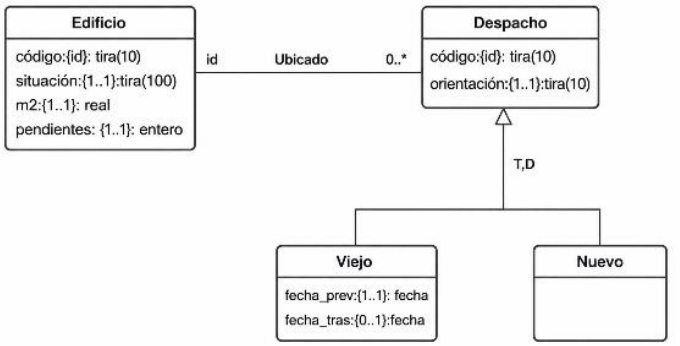
### Q1: Listar edificios ordenados por código.

| Edificio |
|---|
| código:{id}: tira(10)<br>situación:{1..1}:tira(100)<br>m2:{1..1}: real<br>pendientes: {1..1}: entero |

Se aplican las reglas de transformación RT1 (Edificio), RT4 (cod_edif) y RT5 (cod_edif). Se incluyen todos los atributos del edificio.

| edificios | | |
|---|---|---|
| agrupa | int | K |
| cod_edif | text | C↑ |
| situacion | text | |
| m2 | float | |
| pendientes | int | |

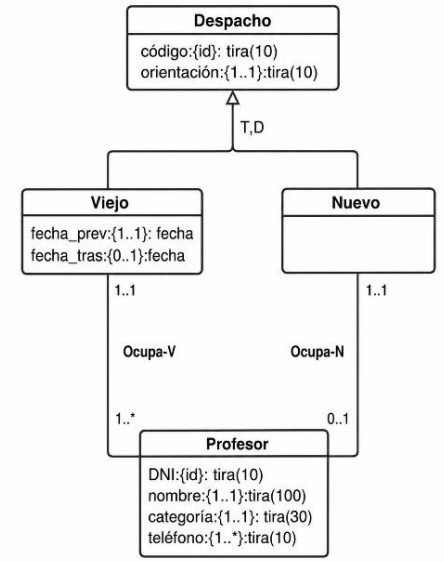
**Q2: Listar despachos viejos pendientes de un edificio, ordenados por fecha prevista.**




Se aplican las reglas de transformación RT1 (Despacho) en la especialización Viejo, RT4 (fecha_prev) y RT5(cod_desp). No se incluyen todos los atributos del edificio ni de Despacho_viejo.

| Edificio X Despacho_viejo | | |
|---|---|---|
| cod_edif | text | K |
| fecha_prev | date | C↑ |
| cod_desp | text | C↑ |
| fecha_tras | date | |
| orientación | text | |

### Q3: Listar profesores de un despacho viejo




Se aplican las reglas de transformación RT1 (Profesor con Ocupa-V), RT2 (búsqueda con igualdad por cod_edif_viejo y cod_desp_viejo), RT4 (nombre) y RT5 (dni). No se incluyen todos los atributos del Profesor.

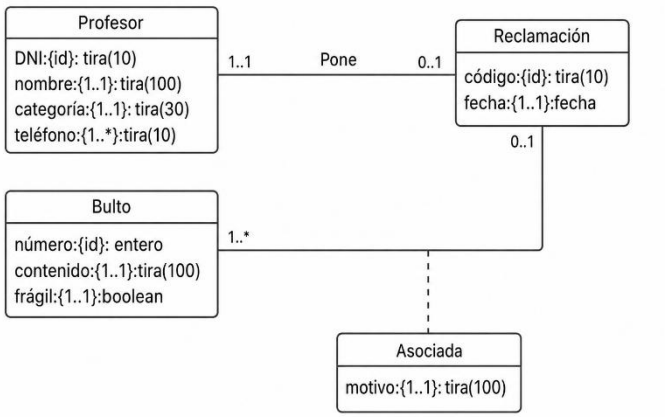
| Despacho_viejo X Profesor | | |
|---|---|---|
| cod_edif_viejo | text | K |
| cod_desp_viejo | text | K |
| nombre | text | C↑ |
| dni | text | C↑ |
| categoria | text | |
| telefonos | text | |
| cod_edif_nuevo | text | |
| cod_desp_nuevo | text | |
| fecha_prev | date | |
| fecha_tras | date | |

### Q4: Consultar los datos completos del profesor elegido.

Se aplican las reglas de transformación RT1 (Profesor) y RT2 (búsqueda con igualdad por dni). Se incluyen todos los atributos del profesor y se añaden los datos necesarios de sus despachos asociados: el despacho viejo que ocupaba, el despacho nuevo que tiene asignado y las fechas previstas y reales del traslado.

| Datos_Profesor | | |
|---|---|---|
| dni | text | K |
| nombre | text | |
| categoria | text | |
| telefonos | text | |
| cod_edif_viejo | text | |
| cod_desp_viejo | text | |
| cod_edif_nuevo | text | |
| cod_desp_nuevo | text | |
| fecha_prev | date | |
| fecha_tras | date | |

### O2: Registrar una reclamación por daños en el traslado




Como se ha decidido representar la asociación Pone entre Profesor y Reclamacion, y la asociación entre Reclamacion y los bultos dañados mediante el UDT Bulto_reclamado, ya tenemos creada la tabla asociada a este acceso.

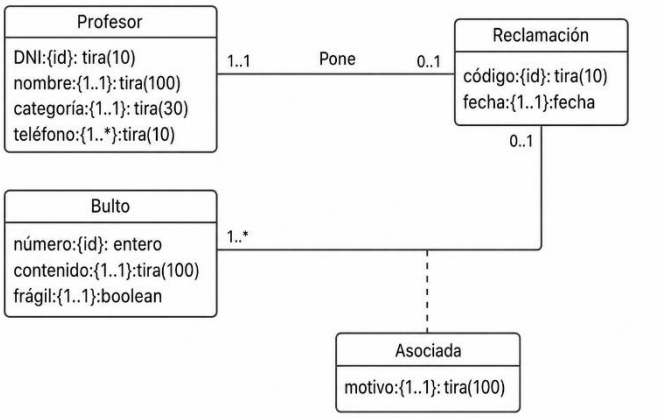
| Reclamacion | | |
|---|---|---|
| cod_rec | text | K |
| dni | text | |
| fecha_rec | date | |
| bultos_rec | {Bulto_reclamado} | |

### Q5: Listar profesores trasladados ordenados por nombre, para elegir uno.

Se aplican las reglas de transformación RT1 (Profesor), RT2 (búsqueda con igualdad por estado), RT4 (nombre) y RT5 (dni). No se incluyen todos los atributos del Profesor. Se incluye un atributo artificial llamado estado que tendrá valor 'TRASLADADO para los profesores que hayan realizado ya el traslado.

| profes_trasladados | | |
|---|---|---|
| estado | text | K |
| nombre | text | C↑ |
| dni | text | C↑ |
| categoria | text | |
| telefonos | text | |
| cod_edif_viejo | text | |
| cod_desp_viejo | text | |
| cod_edif_nuevo | text | |
| cod_desp_nuevo | text | |
| fecha_tras | date | |

### Q6: Consultar si el profesor elegido ya tiene una reclamación registrada.




Se aplican las reglas de transformación RT1 (Reclamación) y RT2 (búsqueda por dni. Se incluyen todos los atributos de Reclamación.

| Profesor x Reclamacion | | |
|---|---|---|
| dni | text | K |
| cod_rec | text | |
| fecha_rec | date | |
| bultos_rec | {Bulto_reclamado} | |

### Q7: Consultar la hoja de traslado del profesor elegido, incluyendo fecha, observaciones y bultos

Esta consulta es la misma que la O1, por tanto ya tenemos la tabla Cassandra adecuada para ella:

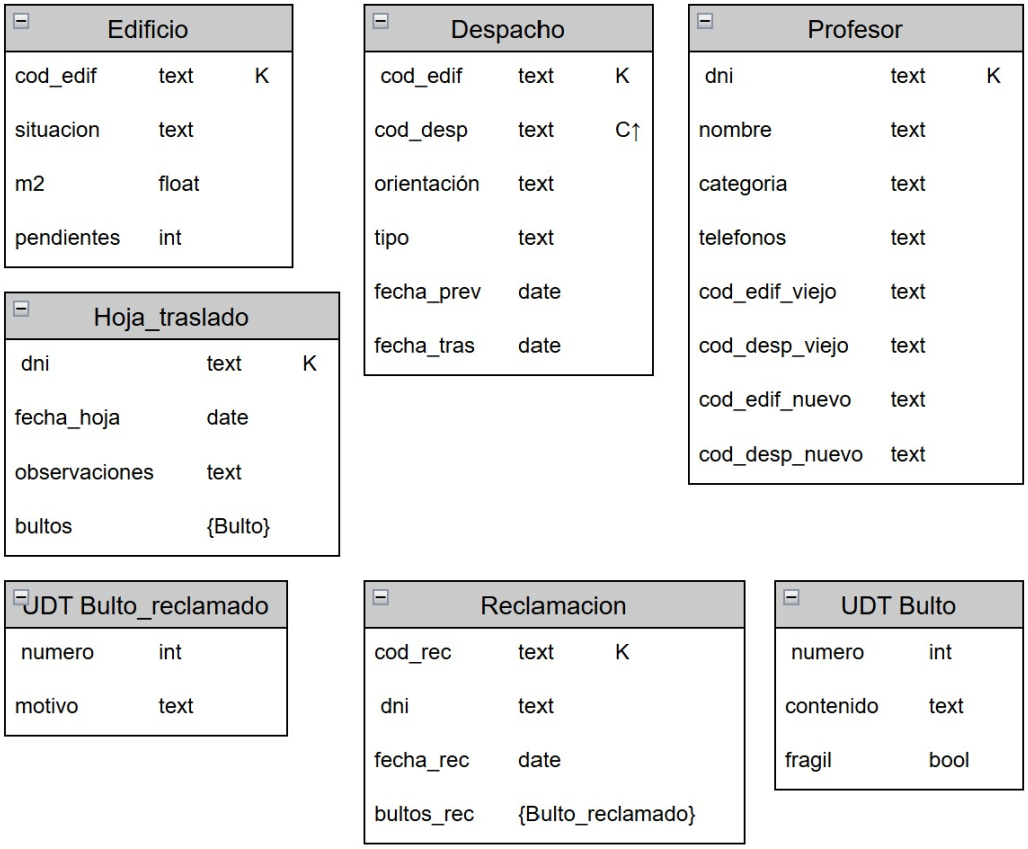
| Hoja_traslado | | |
|---|---|---|
| dni | text | K |
| fecha_hoja | date | |
| observaciones | text | |
| bultos | {Bulto} | |

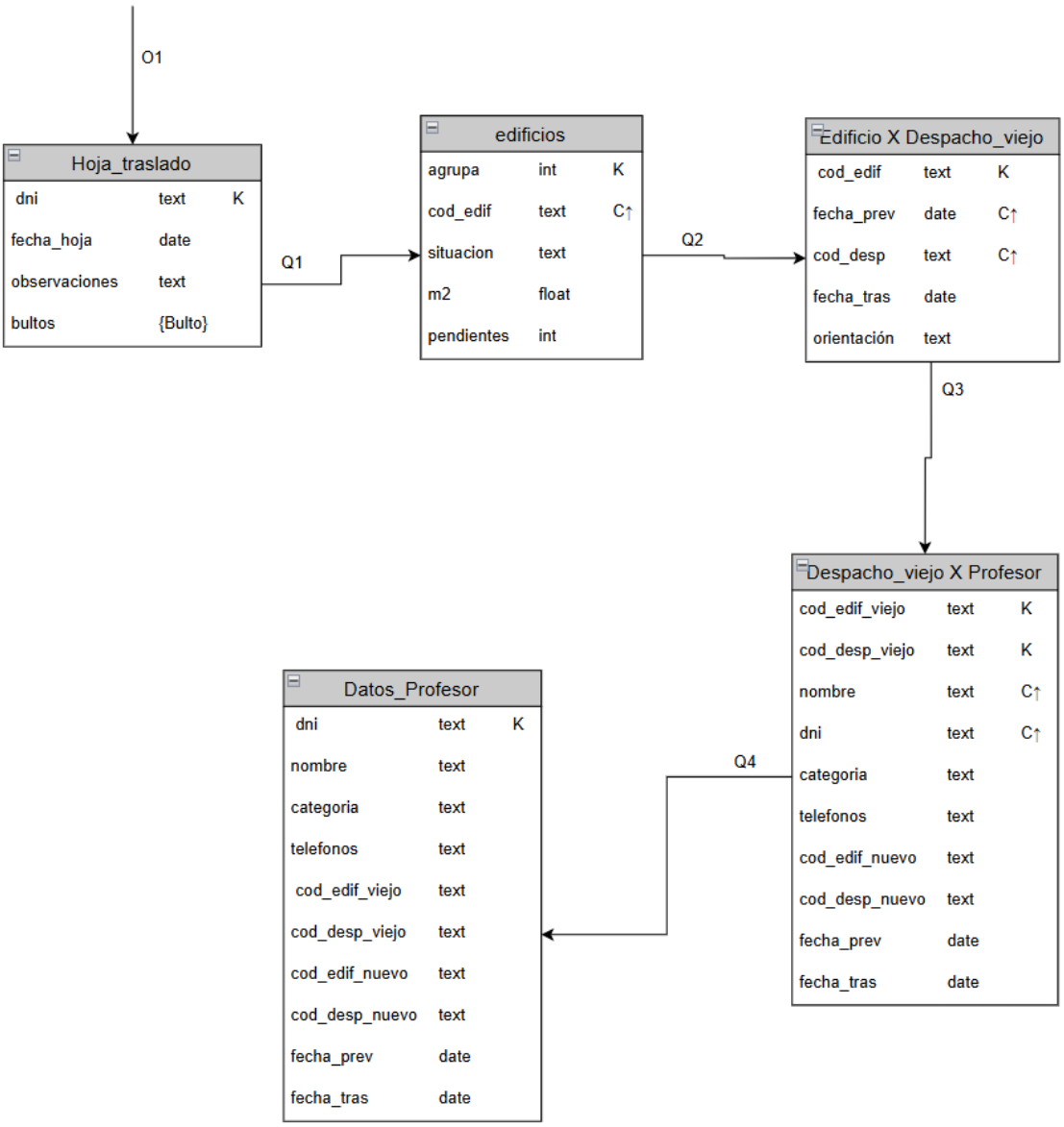
**Q8: Listar los bultos del profesor para elegir cuáles han sufrido daños e indicar el motivo.**

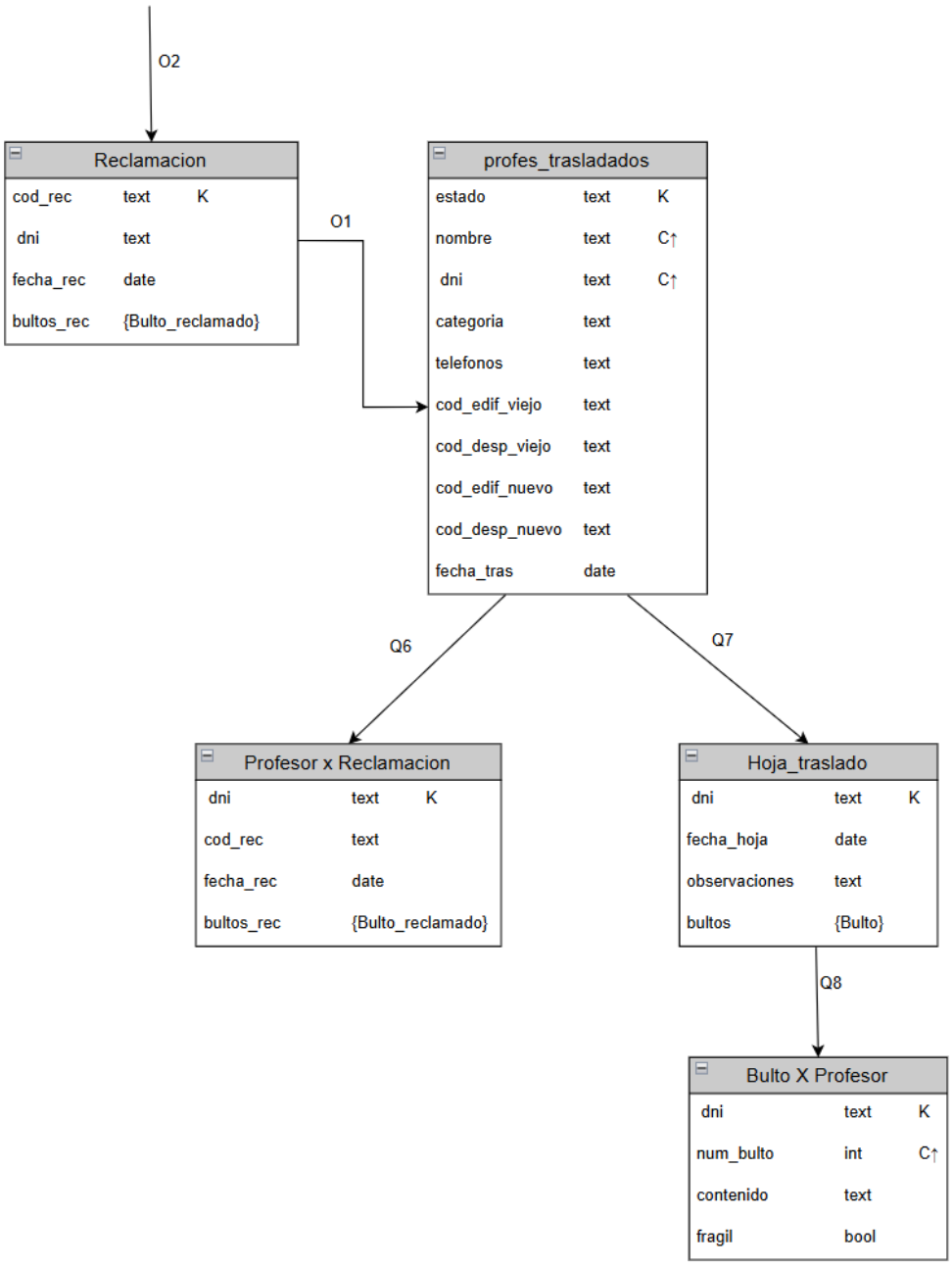
Se aplican las reglas de transformación RT1 (Bulto) , RT2 (búsqueda con igualdad por dni), RT4(num_bulto) y RT5(num_bulto). Se incluyen todos los atributos de UDT Bulto.

| Bulto X Profesor | | |
|---|---|---|
| dni | text | K |
| num_bulto | int | C↑ |
| contenido | text | |
| fragil | bool | |

## 1.2.4 Diagrama Chebotko




O1
Hoja_traslado
dni text K
fecha_hoja date
observaciones text
bultos {Bulto}
Q1
edificios
agrupa int K
cod_edif text C↑
situacion text
m2 float
pendientes int
Q2
Edificio X Despacho_viejo
cod_edif text K
fecha_prev date C↑
cod_desp text C↑
fecha_tras date
orientación text
Q3
Despacho_viejo X Profesor
cod_edif_viejo text K
cod_desp_viejo text K
nombre text C↑
dni text C↑
categoria text
telefonos text
cod_edif_nuevo text
cod_desp_nuevo text
fecha_prev date
fecha_tras date
Q4
Datos_Profesor
dni text K
nombre text
categoria text
telefonos text
cod_edif_viejo text
cod_desp_viejo text
cod_edif_nuevo text
cod_desp_nuevo text
fecha_prev date
fecha_tras date




Las restricciones de integridad que no se han podido expresar en el diagrama Chebotko:

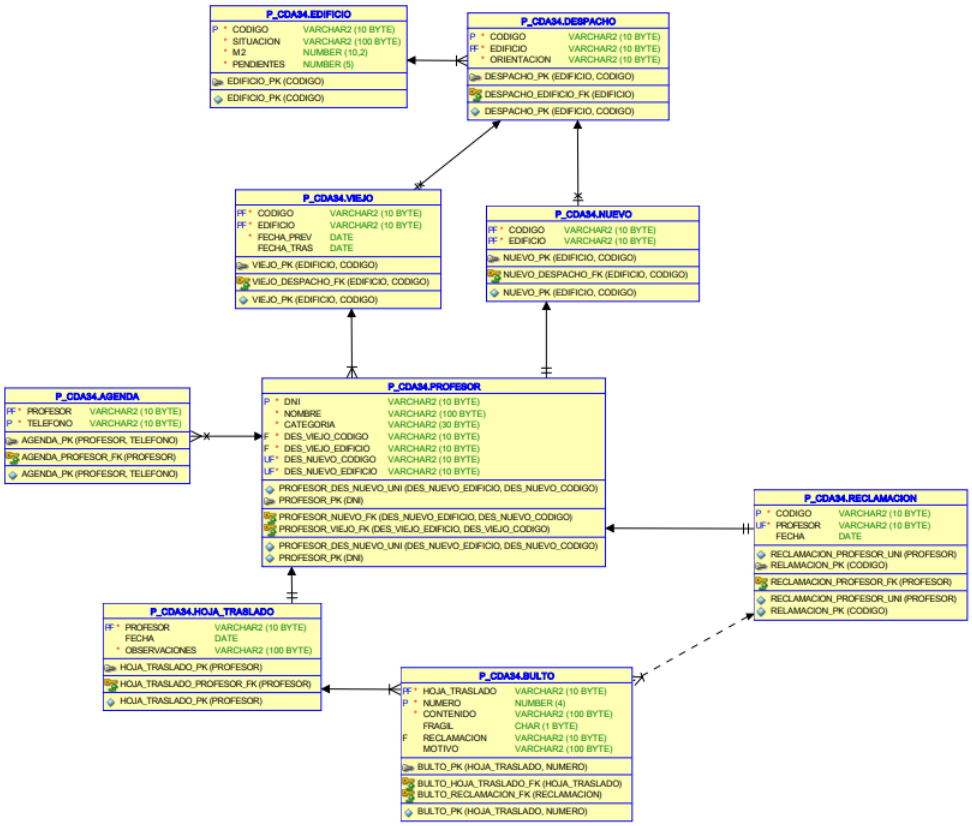
1) En la clase Despacho, el atributo estado solo podrá tomar dos valores ‘TRASLADADO’ y ‘NO TRASLADADO’.
2) Un Profesor solo puede poner una Reclamación.
3) El atributo fecha_tras de la clase Despacho deber ser mayor al atributo fecha_hoja de la clase Hoja_Traslado.
4) El atributo fecha_rec de la clase Reclamación deber ser mayor al atributo fecha_hoja de la clase Hoja_Traslado.
5) (cod_edif_nuevo, cod_desp_nuevo) en Profesor es único.
6) El número de cada Bulto debe ser único dentro de la hoja de traslado.
7) El valor de orientación en Despacho debe pertenecer a {N,S,E,O}.
8) Una reclamación debe incluir al menos un bulto.

9) Los bultos de una reclamación deben ser del profesor que hace la reclamación.
10) Todos los despachos viejos deben estar ocupados al menos por un profesor.
11) Si un bulto aparece en una reclamación, debe obligatoriamente indicarse el motivo.

## 1.3 Tarea 2: Creación y carga de la base de datos Cassandra

### 1.3.1 Diagrama de la base de datos Oracle generado con el Datamodeler

En la figura siguiente se muestra el esquema lógico de la base de datos ya creada en el Oracle.




### 1.3.2 Esquema de la BD en Cassandra

A continuación incluimos las instrucciones de creación del Keyspace y de las tablas en nuestro cluster Cassandra.

#### 1.3.2.1 Creación del *keyspace*

```
CREATE KEYSPACE traslado_dsic

WITH replication ={'class': 'SimpleStrategy',
'replication_factor':'3'}

AND durable_writes=true;
```

#### 1.3.2.2 Creación de tipos definidos por el usuario

```
CREATE TYPE bulto (numero int, contenido text, fragil boolean);

CREATE TYPE bulto_reclamado (numero int, motivo text);
```

#### 1.3.2.3 Creación de table Edificio

```
CREATE TABLE edificio (cod_edif text PRIMARY KEY, situacion text,
m2 float, pendientes int);
```

#### 1.3.2.4 Creación de table Despacho

```
CREATE TABLE despacho (cod_edif text, cod_desp text, orientacion
text, tipo text, fecha_prev date, fecha_tras date, PRIMARY KEY
(cod_edif,cod_desp)) WITH CLUSTERING ORDER BY (cod_desp ASC);
```

#### 1.3.2.5 Creación de table Profesor

```
CREATE TABLE profesor (dni text PRIMARY KEY, nombre text,
categoria text, telefono set<text>,cod_edif_viejo text,
cod_desp_viejo text, cod_edif_nuevo text, cod_desp_nuevo text);
```

#### 1.3.2.6 Creación de table Hoja_traslado

```
CREATE TABLE hoja_traslado (dni text PRIMARY KEY, fecha_hoja
date, observaciones text, bultos set<frozen<bulto>>);
```

#### 1.3.2.7 Creación de table Reclamación

```
CREATE TABLE reclamacion (cod_rec text PRIMARY KEY, dni text,
fecha_rec date, bultos_rec set<frozen<bulto_reclamado>>);
```

#### 1.3.2.8 Creación de table Edificios

```
CREATE TABLE edificios(agrupa int, cod_edif text, situacion text,
m2 float, pendientes int, PRIMARY KEY (agrupa, cod_edif)) WITH
CLUSTERING ORDER BY (cod_edif ASC);
```

#### 1.3.2.9 Creación de table EdificioXdespacho_viejo

```
CREATE TABLE edificioXdespacho_viejo (cod_edif text, fecha_prev
date, cod_desp text, fecha_tras date, orientacion text, PRIMARY
KEY(cod_edif,fecha_prev,cod_desp))WITH CLUSTERING ORDER BY
(fecha_prev ASC, cod_desp ASC);
```

#### 1.3.2.10 Creación de table despacho_viejoXprofesor

```
CREATE TABLE despacho_viejoXprofesor (cod_edif_viejo text,
cod_desp_viejo text, nombre text, dni text, categoria text,
telefono set<text>,cod_edif_nuevo text, cod_desp_nuevo text,
fecha_prev date, fecha_tras date, PRIMARY KEY ((cod_edif_viejo,
cod_desp_viejo), nombre, dni) WITH CLUSTERING ORDER BY (nombre
ASC, dni ASC);
```

#### 1.3.2.11 Creación de table Datos_profesor

```
CREATE TABLE datos_profesor(dni text PRIMARY KEY, nombre text,
categoria text, telefono set<text>,cod_edif_viejo text,
cod_desp_viejo text, cod_edif_nuevo text, cod_desp_nuevo text,
fecha_prev date, fecha_tras date);
```

#### 1.3.2.12 Creación de table Profes_trasladados

```
CREATE TABLE profes_trasladados ( estado text, nombre text, dni
text, categoria text, telefonos set<text>, cod_edif_viejo text,
cod_desp_viejo text, cod_edif_nuevo text, cod_desp_nuevo text,
fecha_tras date,  PRIMARY KEY(estado,nombre,dni)) WITH CLUSTERING
ORDER BY (nombre ASC, dni ASC);
```

#### 1.3.2.13 Creación de table ProfesorXreclamacion

```
CREATE TABLE profesorXreclamacion(dni text PRIMARY KEY, cod_rec
text, fecha_rec date, bultos_rec set<frozen<bulto_reclamado>>);
```

#### 1.3.2.14 Creación de table BultoXprofesor

```
CREATE TABLE bultoXprofesor(dni text, num_bulto int, contenido
text, fragil boolean, PRIMARY KEY(dni,num_bulto)) WITH CLUSTERING
ORDER BY (num_bulto ASC);
```

### 1.3.3 Carga de la BD Cassandra

Para cargar las tablas de la BD Cassandra a partir de nuestra base de datos relacional en Oracle tenemos que crear una consulta en Oracle que genere la información que hay que cargar en cada tabla Cassandra. Una vez creada la consulta, la ejecutaremos y exportaremos la información a un fichero *csv* que luego trasladaremos a la máquina virtual e importaremos en el *cqlsh* de Cassandra.

#### 1.3.3.1 Tabla Edificio

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la que se muestra a continuación. Se hace uso de la función to_char en la columna m2 para garantizar que el carácter de separación de los decimales sea el punto (".") en lugar de la coma (","):

```
select codigo cod_edif, situacion, to_char(m2,'999999.99') m2,
pendientes
from p_cda34.edificio;
```

Exportando el resultado de esta consulta al fichero edificio.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY edificio (cod_edif, situacion, m2, pendientes) FROM
'edificio.csv' WITH HEADER = TRUE;
```

#### 1.3.3.2 Tabla Despacho

La consulta en Oracle que devuelve las filas que habrá que insertar en la tabla es la siguiente. En esta consulta se hace uso de la sentencia case para asignar a los atributos que estaban en las clases especializadas los valores adecuados cuando existen o el valor null cuando la propiedad no es aplicable. Además, cambiamos el formato de las fechas al formato de Cassandra (YYYY-MM-DD).

```
select d.edificio cod_edif, d.codigo cod_desp, d.orientacion,

    case when exists (select * from p_cda34.viejo v where
v.edificio = d.edificio and v.codigo= d.codigo) then 'Viejo' else
'Nuevo' end tipo,

    case when exists (select * from p_cda34.viejo v where
v.edificio = d.edificio and v.codigo= d.codigo) then (select
to_char(v.fecha_prev,'YYYY-MM-DD') from p_cda34.viejo v where
v.edificio = d.edificio and v.codigo= d.codigo) else null end
fecha_prev,
```

```
    case when exists (select * from p_cda34.viejo v where
v.edificio = d.edificio and v.codigo= d.codigo) then (select
to_char(v.fecha_tras,'YYYY-MM-DD') from p_cda34.viejo v where
v.edificio = d.edificio and v.codigo= d.codigo) else null end
fecha_tras

from p_cda34.despacho d;
```

Exportando el resultado de la consulta al fichero despacho.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY despacho (cod_edif, cod_desp, orientacion, tipo, fecha_prev,
fecha_tras) FROM 'despacho.csv' WITH HEADER = TRUE;
```

#### 1.3.3.3 Tabla Profesor

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la que se muestra a continuación. En esta consulta tenemos que generar un valor de tipo conjunto (set<text>) para los teléfonos, de forma que Cassandra lo cargue adecuadamente. Para lograrlo, aparece una subconsulta en la cláusula from junto con la función listagg de Oracle, que genera una lista de valores separados por comas a la cual se le añaden las llaves { y } para completar el formato de conjunto correctamente.

```
select p.dni,p.nombre, p.categoria, temp_telf.telefonos,
p.des_viejo_edificio,
        p.des_viejo_codigo, p.des_nuevo_edificio,
p.des_nuevo_codigo
from p_cda34.profesor p
join
    (select t.profesor, '{''' || listagg(t.telefono,''',''')
        within group (order by t.telefono) ||'''}' telefonos
        from p_cda34.agenda t group by t.profesor) temp_telf
on p.dni=temp_telf.profesor;
```

Exportando el resultado de la consulta al fichero profesor.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY profesor ( dni, nombre, categoria, telefono,
cod_edif_viejo, cod_desp_viejo,     cod_edif_nuevo,
cod_desp_nuevo ) FROM 'profesor.csv' WITH HEADER = TRUE;
```

#### 1.3.3.4 Tabla Hoja_traslado

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la siguiente. En este caso, la presencia de una fecha nos obliga a generar los datos mediante una select que cambia su formato al formato de Cassandra (YYYY-MM-DD). Además, se hace uso de la sentencia case para asignar al atributo fragil, que en Cassandra es de tipo boolean (que no existe en Oracle), el valor true/false. Finalmente, la generación del conjunto para la columna bultos se realiza en la propia instrucción select mediante la función listagg de Oracle.

```
select h.profesor dni,
to_char(h.fecha, 'YYYY-MM-DD') fecha_hoja,
h.observaciones,temp_bultos.bultos
from p_cda34.hoja_traslado h
join
    (select b.hoja_traslado, '{' || listagg('{numero:'||b.numero
|| ',contenido:'''||
                    b.contenido||''''||
                    ',fragil:'|| case when b.fragil='S'
```

```
                    then 'true' else 'false' end || '}',',')
        within group (order by b.numero) ||'}' bultos
        from p_cda34.bulto b group by b.hoja_traslado)
temp_bultos
on h.profesor=temp_bultos.hoja_traslado;
```

Exportando el resultado de la select al fichero hoja_traslado.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY hoja_traslado (dni, fecha_hoja, observaciones, bultos)  FROM
'hoja_traslado.csv' WITH HEADER = TRUE;
```

#### 1.3.3.5 Tabla Reclamación

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la siguiente. En esta consulta modificamos el formato de la fecha para adaptarlo al de Cassandra (YYYY-MM-DD). Además, utilizamos una subconsulta en la cláusula from junto con la función listagg para generar directamente el conjunto de datos de la columna bultos_rec, asegurándonos de filtrar previamente aquellos registros que no tienen una reclamación asociada (is not null).

```
select r.codigo cod_rec, r.profesor dni, to_char(r.fecha, 'YYYY-
MM-DD') fecha_rec,
        temp_bultos.bultos_rec
from p_cda34.reclamacion r
join
    (select b.reclamacion, '{' || listagg('{numero:'||b.numero
||',motivo:'''||b.motivo||''''||'}',',')
        within group (order by b.numero) ||'}' bultos_rec
        from p_cda34.bulto b where b.reclamacion is not null
        group by b.reclamacion) temp_bultos
on r.codigo=temp_bultos.reclamacion;
```

Exportando el resultado de la consulta al fichero reclamacion.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY reclamacion (cod_rec, dni, fecha_rec, bultos_rec) FROM
'reclamacion.csv' WITH HEADER = TRUE;
```

#### 1.3.3.6 Tabla Edificios

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la siguiente. En esta consulta asignamos al atributo agrupa el valor 1 para todas las filas que devuelve la consulta. Además, el uso de la función to_char en la columna m2 sirve para que el carácter de separación de los decimales sea el punto (".") en lugar de la coma.

```
select 1 agrupa,codigo cod_edif, situacion,
to_char(m2,'999999.99') m2, pendientes
from p_cda34.edificio;
```

Exportando el resultado de la select al fichero edificios.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY edificios (agrupa, cod_edif, situacion, m2, pendientes) FROM
'edificios.csv' WITH HEADER = TRUE;
```

#### 1.3.3.7 Tabla EdificioXdespacho_viejo

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la siguiente. En este caso, la consulta concatena las tablas Viejo y Despacho. Además, volvemos a utilizar la función to_char para adaptar el formato de las fechas al estándar de Cassandra (YYYY-

MM-DD) y aplicamos una condición en la cláusula where para filtrar y obtener únicamente aquellos registros donde la fecha de traslado (fecha_tras) sea nula.

```
select v.edificio cod_edif, to_char(v.fecha_prev,'YYYY-MM-DD'),
v.codigo cod_desp,
    to_char(v.fecha_tras,'YYYY-MM-DD'), d.orientacion
from p_cda34.viejo v, p_cda34.despacho d
where v.edificio=d.edificio and v.codigo=d.codigo and
v.fecha_tras is null;
```

Exportando el resultado de la consulta al fichero edificioXdespachoViejo.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY edificioxdespacho_viejo(cod_edif, fecha_prev, cod_desp,
fecha_tras, orientacion) FROM 'edificioXdespachoViejo.csv'  WITH
HEADER = TRUE;
```

#### 1.3.3.8 Tabla Despacho_viejoXprofesor

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la siguiente. En esta consulta se concatenan varias tablas mediante la cláusula join. Además, volvemos a modificar el formato de las fechas para adaptarlo al estándar de Cassandra (YYYY-MM-DD) y utilizamos una subconsulta con la función listagg de Oracle para generar correctamente el tipo conjunto (set<text>) para la columna de teléfonos.

```
select p.des_viejo_edificio, p.des_viejo_codigo, p.nombre, p.dni
,p.categoria,
    temp_telf.telefonos,p.des_nuevo_edificio, p.des_nuevo_codigo,
    to_char(v.fecha_prev,'YYYY-MM-DD')
fecha_prev,to_char(v.fecha_tras,'YYYY-MM-DD')fecha_tras
from p_cda34.profesor p
join p_cda34.viejo v on p.des_viejo_edificio= v.edificio and
p.des_viejo_codigo=v.codigo
join
    (select t.profesor, '{''' || listagg(t.telefono,''',''')
        within group (order by t.telefono) ||'''}' telefonos
        from p_cda34.agenda t group by t.profesor) temp_telf
on p.dni=temp_telf.profesor;
```

Exportando el resultado de la consulta al fichero despacho_viejoXprofesor.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY despacho_viejoXprofesor(     cod_edif_viejo, cod_desp_viejo,
nombre, dni, categoria, telefono,     cod_edif_nuevo,
cod_desp_nuevo, fecha_prev, fecha_tras ) FROM
'despacho_viejoXprofesor.csv' WITH HEADER = TRUE;
```

#### 1.3.3.9 Tabla Datos_profesor

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la siguiente. En esta consulta se concatenan las tablas correspondientes mediante la cláusula join. Además, se modifica el formato de las fechas para adaptarlo al estándar de Cassandra (YYYY-MM-DD) y se utiliza una subconsulta con la función listagg de Oracle para generar de manera correcta el tipo conjunto (set<text>) para la columna de teléfonos.

```
select p.dni,
p.nombre,p.categoria,temp_telf.telefonos,p.des_viejo_edificio,
        p.des_viejo_codigo,p.des_nuevo_edificio,
p.des_nuevo_codigo,
    to_char(v.fecha_prev,'YYYY-MM-DD')
fecha_prev,to_char(v.fecha_tras,'YYYY-MM-DD')fecha_tras
```

```
from p_cda34.profesor p
join p_cda34.viejo v on p.des_viejo_edificio= v.edificio and
p.des_viejo_codigo=v.codigo
join
    (select t.profesor, '{''' || listagg(t.telefono,''',''')
        within group (order by t.telefono) ||'''}' telefonos
        from p_cda34.agenda t group by t.profesor) temp_telf
on p.dni=temp_telf.profesor;
```

Exportando el resultado de la consulta al fichero datos_profesor.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY datos_profesor(dni, nombre, categoria,
telefono,cod_edif_viejo, cod_desp_viejo, cod_edif_nuevo,
cod_desp_nuevo, fecha_prev, fecha_tras) FROM 'datos_profesor.csv'
WITH HEADER = TRUE;
```

#### 1.3.3.10 Tabla Profes_trasladados

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la siguiente. En esta consulta se hace uso de la sentencia case para evaluar si el profesor ya ha sido trasladado o no, devolviendo una cadena de texto descriptiva según corresponda. Asimismo, se utiliza la función to_char para adaptar el formato de la fecha de traslado al estándar de Cassandra (YYYY-MM-DD) y se emplea una subconsulta junto con la función listagg para generar adecuadamente el tipo conjunto (set<text>) destinado a la columna de teléfonos.

```
select case when exists(select * from p_cda34.viejo v
                            where v.edificio=p.des_viejo_edificio
and
                                v.codigo=p.des_viejo_codigo and
                                v.fecha_tras is not null)
                                then 'TRASLADADO' else 'NO
TRASLADADO' end estado,
        p.nombre, p.dni, p.categoria, temp_telf.telefonos,
p.des_viejo_edificio,
        p.des_viejo_codigo,p.des_nuevo_edificio,
p.des_nuevo_codigo,
        to_char(v.fecha_tras,'YYYY-MM-DD')fecha_tras
from p_cda34.profesor p

join p_cda34.viejo v on p.des_viejo_edificio= v.edificio and
p.des_viejo_codigo=v.codigo
join
    (select t.profesor, '{''' || listagg(t.telefono,''',''')
        within group (order by t.telefono) ||'''}' telefonos
        from p_cda34.agenda t group by t.profesor) temp_telf
on p.dni=temp_telf.profesor;
```

Exportando el resultado de la consulta al fichero profes_trasladados.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY profes_trasladados (estado, nombre, dni, categoria,
telefonos, cod_edif_viejo, cod_desp_viejo, cod_edif_nuevo,
cod_desp_nuevo, fecha_tras) FROM 'profes_trasladados.csv'WITH
HEADER = TRUE;
```

#### 1.3.3.11 Tabla ProfesorXreclamacion

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la que se muestra a continuación. En ella se adaptan las fechas al formato de Cassandra (YYYY-MM-DD) y

se emplea una subconsulta junto con la función listagg para agrupar y generar de forma directa el conjunto de tipo UDT (set<frozen<...>>) correspondiente a la columna bultos_rec.

```
select r.profesor dni, r.codigo cod_rec,to_char(r.fecha,'YYYY-MM-
DD'),
        temp_bultos.bultos_rec
from p_cda34.reclamacion r
join
     (select b.reclamacion, '{' || listagg('{numero:'||b.numero
||',motivo:'''||b.motivo||''''||'}',',')
        within group (order by b.numero) ||'}' bultos_rec
        from p_cda34.bulto b where b.reclamacion is not null
        group by b.reclamacion) temp_bultos
on r.codigo=temp_bultos.reclamacion;
```

Exportando el resultado de la select al fichero ProfesorXreclamacion.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY profesorXreclamacion (dni, cod_rec, fecha_rec, bultos_rec)
FROM 'ProfesorXreclamacion.csv' WITH HEADER = TRUE;
```

#### 1.3.3.12 Tabla BultoXprofesor

La consulta en Oracle que nos devuelve las filas que habrá que insertar en la tabla es la que se muestra a continuación. En esta consulta se hace uso de la sentencia case para transformar el valor del atributo fragil (que en Oracle se almacena como 'S') al formato booleano (true/false) requerido por Cassandra.

```
select hoja_traslado dni,numero num_bulto,contenido,
        case when fragil='S' then 'true' else 'false' end fragil
from p_cda34.bulto;
```

Exportando el resultado de la select al fichero bultoXprofesor.csv, la instrucción de carga en Cassandra es la siguiente:

```
COPY bultoXprofesor (dni, num_bulto, contenido, fragil) FROM
'bultoXprofesor.csv' WITH HEADER = TRUE;
```

### 1.3.4 Consultas sobre Cassandra

1) Q1: Listar edificios ordenados por código

```
cqlsh:traslado_dsic> SELECT cod_edif, situacion, m2, pendientes FROM
edificios WHERE agrupa = 1;
```

| cod_edif | situacion | m2 | pendientes |
|---|---|---|---|
| E01 | Situacion_1 | 1574.48999 | 0 |
| E02 | Situacion_2 | 644.95001 | 0 |
| E03 | Situacion_3 | 928.77002 | 0 |
| E04 | Situacion_4 | 618.10999 | 0 |
| E05 | Situacion_5 | 540.06 | 0 |
| E06 | Situacion_6 | 576.44 | 0 |
| E07 | Situacion_7 | 1780.27002 | 0 |
| E08 | Situacion_8 | 1981.28003 | 299 |
| E09 | Situacion_9 | 848.91998 | 0 |
| E10 | Situacion_10 | 1970.16003 | 0 |

2) Q2: Listar despachos viejos pendientes de un edificio, ordenados por fecha prevista.

```
cqlsh:traslado_dsic> SELECT cod_edif, fecha_prev, cod_desp,
fecha_tras, orientacion FROM edificioXdespacho_viejo WHERE cod_edif =
'E08' limit 10;
```

| cod_edif | fecha_prev | cod_desp | fecha_tras | orientacion |
|---|---|---|---|---|
| E08 | 2020-01-01 | D0438 | null | ESTE |
| E08 | 2020-01-15 | D0405 | null | SUR |
| E08 | 2020-01-18 | D0463 | null | OESTE |
| E08 | 2020-02-01 | D0234 | null | ESTE |
| E08 | 2020-02-15 | D0201 | null | SUR |
| E08 | 2020-03-27 | D0083 | null | OESTE |
| E08 | 2020-04-01 | D0403 | null | OESTE |
| E08 | 2020-04-11 | D0165 | null | SUR |
| E08 | 2020-04-21 | D0171 | null | OESTE |
| E08 | 2020-05-17 | D0285 | null | SUR |

3) Q3: Listar profesores de un despacho viejo.

```
cqlsh:traslado_dsic> SELECT cod_edif_viejo, cod_desp_viejo,
nombre,dni,fecha_prev, fecha_tras FROM despacho_viejoXprofesor WHERE
cod_edif_viejo= 'E08'   AND cod_desp_viejo = 'D0249';
```

| cod_edif_viejo | cod_desp_viejo | nombre | dni | fecha_prev | fecha_tras |
|---|---|---|---|---|---|
| E08 | D0249 | Profesor_1 | P0001 | 2020-08-11 | 2022-09-18 |

4) Q4: Consultar los datos completos del profesor elegido.

```
cqlsh:traslado_dsic> SELECT * FROM datos_profesor WHERE dni = 'P0001';
```

| dni | categoria | cod_desp_nuevo | cod_desp_viejo | cod_edif_nuevo | cod_edif_viejo | fecha_prev | fecha_tras | nombre | telefono |
|---|---|---|---|---|---|---|---|---|---|
| P0001 | ASOCIADO | D0364 | D0249 | E08 | E08 | 2020-08-11 | 2022-09-18 | Profesor_1 | {'600000200'} |

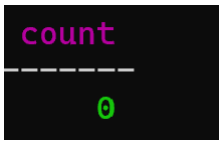
5) Q5: Listar profesores trasladados ordenados por nombre, para elegir uno.

```
cqlsh:traslado_dsic> SELECT * FROM profes_trasladados WHERE estado =
'TRASLADADO' ORDER BY nombre limit 10;
```

| estado | nombre | dni | categoria | cod_desp_nuevo | cod_desp_viejo | cod_edif_nuevo | cod_edif_viejo | fecha_tras | telefonos |
|---|---|---|---|---|---|---|---|---|---|
| TRASLADADO | Profesor_1 | P0001 | ASOCIADO | D0364 | D0249 | E08 | E08 | 2022-09-18 | {'600000200'} |
| TRASLADADO | Profesor_10 | P0010 | ASOCIADO | D0190 | D0009 | E08 | E08 | 2023-05-13 | {'600000009', '600000209'} |
| TRASLADADO | Profesor_103 | P0103 | ASOCIADO | D0429 | D0299 | E08 | E08 | 2022-07-16 | {'600000102'} |
| TRASLADADO | Profesor_104 | P0104 | AYUDANTE | D0187 | D0055 | E08 | E08 | 2022-07-24 | {'600000103'} |
| TRASLADADO | Profesor_106 | P0106 | ASOCIADO | D0160 | D0273 | E08 | E08 | 2022-12-04 | {'600000105'} |
| TRASLADADO | Profesor_107 | P0107 | AYUDANTE | D0486 | D0281 | E08 | E08 | 2022-11-27 | {'600000106'} |
| TRASLADADO | Profesor_108 | P0108 | TITULAR | D0135 | D0021 | E08 | E08 | 2022-05-09 | {'600000107'} |
| TRASLADADO | Profesor_109 | P0109 | ASOCIADO | D0474 | D0251 | E08 | E08 | 2022-01-05 | {'600000108'} |
| TRASLADADO | Profesor_11 | P0011 | AYUDANTE | D0150 | D0027 | E08 | E08 | 2022-10-14 | {'600000010', '600000210'} |
| TRASLADADO | Profesor_111 | P0111 | TITULAR | D0402 | D0206 | E08 | E08 | 2022-05-09 | {'600000110'} |

6) Q6: Consultar si el profesor elegido ya tiene una reclamación registrada.

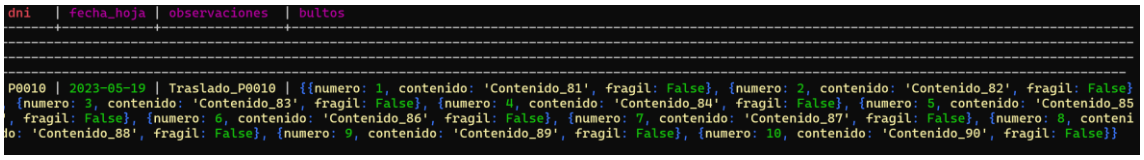
```
cqlsh:traslado_dsic> select count(*) from profesorXreclamacion where
dni='P0010';
```



```
 count
-------
     0
```

7) Q7: Consultar la hoja de traslado del profesor elegido, incluyendo fecha, observaciones y bultos

```
cqlsh:traslado_dsic> SELECT dni, fecha_hoja,observaciones, bultos FROM
hoja_traslado WHERE dni= 'P0010';
```



```
 dni   | fecha_hoja | observaciones | bultos
-------+------------+---------------+--------------------
```

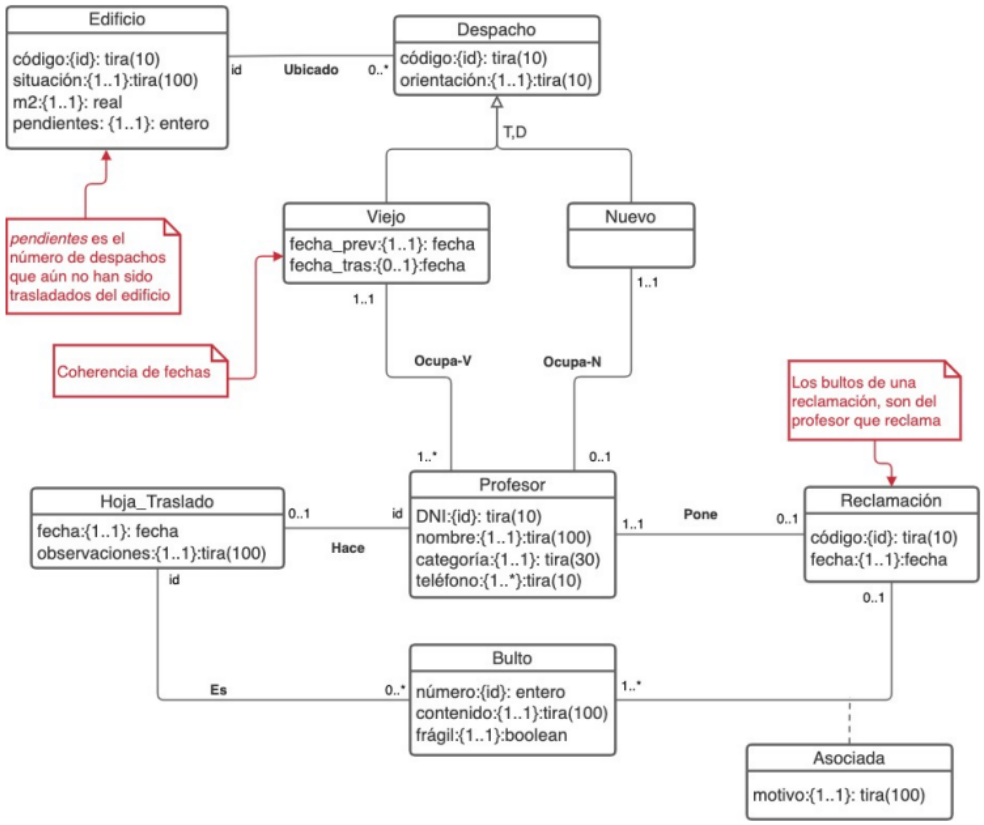
# 2 PARTE 2: NEO4J

## 2.1 Tarea 3: Diseño lógico

### 2.1.1 Descripción final del SI: Compañía de Alquiler de Vehículos

El DSIC con el fin de optimizar el cambio a su nueva sede departamental ha decidido diseñar una base de datos para facilitar la mudanza. El departamento dispone de despachos que se identifican por un código que es único dentro de cada edificio de los que dispone. Los edificios del departamento se identifican por un código único dentro de la UPV y se conoce su situación y su superficie en metros cuadrados. De los despachos interesa saber la orientación (N, S, E, W) y también cuántos despachos están aún pendientes de traslado. La mudanza se realiza de los despachos viejos a los nuevos. Cada profesor, del cual se conoce su DNI, que lo identifica, su nombre, uno o varios teléfonos, su categoría laboral, y el despacho viejo que ocupaba, tiene asignado un despacho nuevo. Los despachos viejos, generalmente, estaban compartidos por más de un profesor y todos estaban ocupados. Los nuevos, por el contrario, son despachos individuales y hay algunos, que por el momento, pueden quedar libres. Se ha elaborado una planificación en la que se indica el día previsto en que se realizará el traslado de un despacho viejo a uno nuevo. Todos los profesores de un despacho se trasladan el mismo día. Como puede haber retrasos, en el momento en que se realice realmente el traslado se debe almacenar el día exacto del mismo. Antes del traslado, cada profesor debe presentar en la secretaría un documento llamado “hoja de traslado” en el que debe figurar la fecha de entrega de la hoja, el profesor que la presenta y las observaciones que considere oportunas para la empresa de mudanzas. Además, dicha hoja debe incluir una lista de los bultos que se van a trasladar. Cada bulto se identifica por un número que es único dentro de cada hoja y se debe indicar claramente su contenido y si es frágil o no. Es posible que algún bulto resulte dañado en el traslado, si esto ocurre, un profesor puede presentar en la secretaría una reclamación (como mucho) en la que debe especificar cuáles de sus bultos han sufrido daños y de qué tipo (pérdida, rotura, …). A cada reclamación, se le asignará un código único en el momento en que se entrega y se guardará la fecha en que se presentó.

## 2.1.2 Diagrama de clases final




Las restricciones de integridad que no se han podido expresar en :

1) El atributo orientación de la clase Edificio toma un valor entre {N,S,W,E}.

2) El atributo fecha_prev de la subclase Viejo deber ser mayor al atributo fecha_entrega de la clase Hoja_Traslado.

3) El atributo fecha_realizado de la subclase Viejo deber ser mayor al atributo fecha_entrega de la clase Hoja_Traslado.

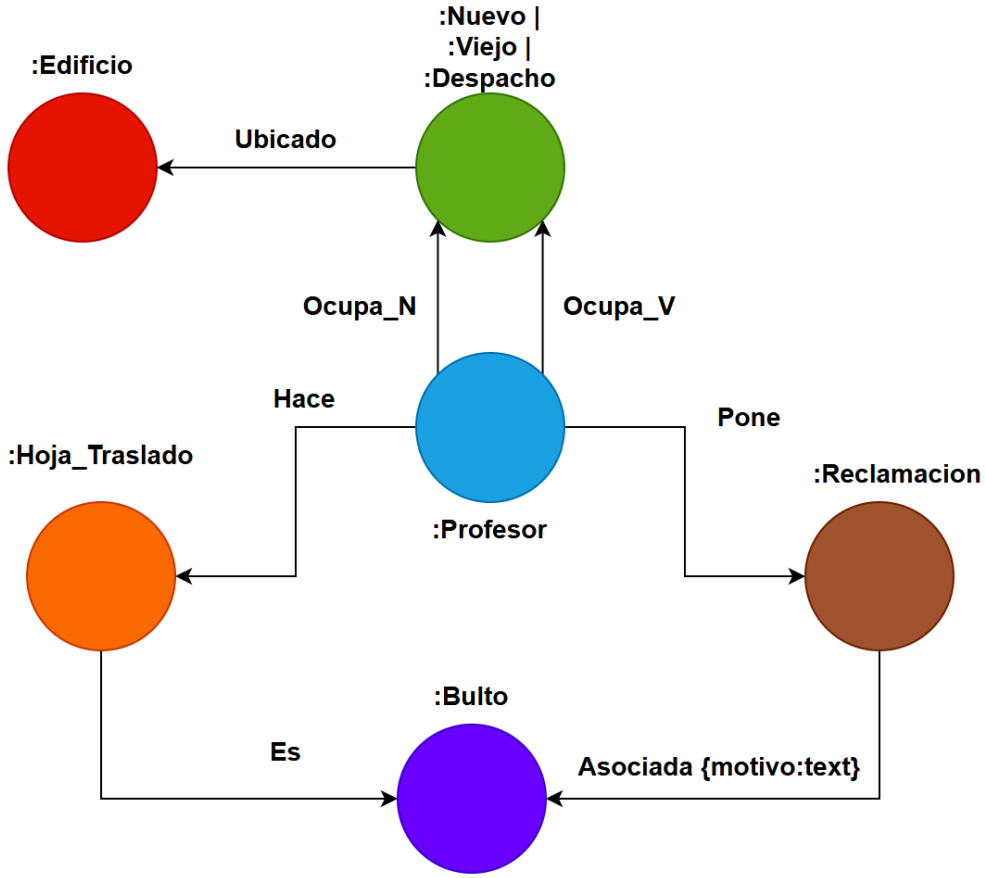
4) El atributo desp_nuevos_pendientes de Edificio es un atributo derivado que se mantendrá automáticamente por el sistema y su valor será en todo momento la cantidad de despachos nuevos que están pendientes de traslado de cada edificio.

5) El atributo desp_viejos_pendientes de Edificio es un atributo derivado que se mantendrá automáticamente por el sistema y su valor será en todo momento la cantidad de despachos viejos que están pendientes de traslado de cada edificio, es decir todos aquellos despachos que el atributo fecha_realizado sea nulo.

6) El atributo fecha_rec de la clase Reclamación debe ser mayor al atributo fecha_realizado de la subclase Viejo.

### 2.1.3 Esquema de Neo4j

Partiendo del diagrama de clases de problema se diseña el esquema Neo4j:




**Propiedades de los nodos:**

- Nodos con etiqueta :Edificio
  - cod_edif: texto
  - situacion: texto
  - m2: real
  - pendientes: entero

- Nodos con etiqueta :Despacho:Nuevo
  - cod_desp: texto
  - orientacion: texto

- Nodos con etiqueta :Despacho:Viejo
  - cod_desp: texto
  - orientacion: texto
  - fecha_prev: fecha
  - fecha_tras: fecha

- Nodos con etiqueta :Profesor
  - dni: texto
  - nombre: texto

categoria: texto
telefono: lista de texto

- Nodos con etiqueta :Hoja_Traslado
  fecha: fecha
  observaciones: texto

- Nodos con etiqueta :Bulto
  numero: entero
  contenido: texto
  fragil: booleano

- Nodos con etiqueta :Reclamacion
  cod_rec: texto
  fecha: fecha

### Propiedades de las relaciones:

- Relaciones del tipo Asociada:
  motivo: texto

### Restricciones

Como en la versión Community de Neo4j sólo se puede definir una restricción de unicidad sobre una o varias propiedades de nodos, vamos a definir las restricciones de unicidad necesarias en nuestro caso:

- propiedad cod_edif de los nodos Edificio.
- propiedad dni de los nodos Profesor.
- propiedad profesor de los nodos Hoja_Traslado.
- propiedad cod_rec de los nodos Reclamacion.

No se define restricción de unicidad simple sobre cod_desp en Despacho ni sobre número en Bulto, porque su identificación depende de la relación con otro nodo.
El resto de las restricciones del diagrama de clases quedarían por expresar:

- Cardinalidad mínima 1 del atributo cod_edif de la clase Edificio.
- Cardinalidad mínima 1 del atributo situacion de la clase Edificio.
- Cardinalidad mínima 1 del atributo m2 de la clase Edificio.
- Cardinalidad mínima 1 del atributo pendientes de la clase Edificio.
- El atributo pendientes de Edificio debe coincidir con el número de despachos que aún no han sido trasladados del edificio.
- Restricción de identificación de la clase Edificio en la asociación Ubicado.
- Restricción de identificación de la clase Despacho en la asociación Ubicado: no pueden existir dos nodos Despacho conectados al mismo nodo Edificio mediante Ubicado.
- Cardinalidad mínima 1 del atributo cod_desp de la clase Despacho.
- Cardinalidad mínima 1 del atributo orientación de la clase Despacho.
- El atributo orientacion de la clase Despacho sólo puede tomar los valores N, S, E o W.
- Cardinalidad mínima 1 del atributo fecha_prev de la clase Viejo.
- Un despacho nuevo como máximo puede estar asignado a un profesor.

- Los despachos viejos estaban ocupados por uno o más profesores.
- Tipo de especialización T, D de la clase Despacho en las clases Nuevo y Viejo.
- Todos los profesores de un mismo despacho viejo se trasladan el mismo día.
- Correcta definición de la asociación Ubicado, o sea, que no pueda haber más de una instancia de la relación Ubicado que conecten a un nodo Despacho y a un nodo Edificio.
- Cardinalidad mínima 1 del atributo dni de la clase Profesor.
- Cardinalidad mínima 1 del atributo nombre de la clase Profesor.
- Cardinalidad mínima 1 del atributo categoria de la clase Profesor.
- Cardinalidad mínima 1 del atributo telefono de la clase Profesor.
- Correcta definición de la asociación Ocupa-V: o sea, que no pueda haber más de una instancia de la relación Ocupa-V que conecten a un nodo Despacho y a un nodo Profesor.
- Correcta definición de la asociación Ocupa-N: o sea, que no pueda haber más de una instancia de la relación Ocupa-N que conecten a un nodo Despacho y a un nodo Profesor.
- Cardinalidad mínima 1 del atributo fecha de la clase Hoja_Traslado.
- Cardinalidad mínima 1 del atributo observaciones de la clase Hoja_Traslado.
- Restricción de identificación de la clase Profesor en la asociación Hace.
- Correcta definición de la asociación Hace: o sea, que no pueda haber más de una instancia de la relación Hace que conecten a un nodo Hoja_Traslado y a un nodo Profesor
- Cardinalidad máxima 1 de Hoja_Traslado en la asociación Hace, es decir, un profesor puede presentar como mucho una hoja de traslado.
- Cardinalidad mínima 1 del atributo numero de la clase Bulto.
- Cardinalidad mínima 1 del atributo contenido de la clase Bulto.
- Cardinalidad mínima 1 del atributo fragil de la clase Bulto.
- Restricción de identificación de la clase Hoja_Traslado en la asociación Es.
- Restricción de identificación de la clase Bulto en la asociación Es:
  no pueden existir dos nodos Bulto conectados a la misma Hoja_Traslado mediante Es que tengan el mismo valor de número.
- Correcta definición de la asociación Es: o sea, que no pueda haber más de una instancia de la relación Es que conecten a un nodo Hoja_Traslado y a un nodo Bulto.
- Cardinalidad mínima 1 del atributo cod_rec de la clase Reclamacion.
- Cardinalidad mínima 1 del atributo fecha de la clase Reclamacion.
- Correcta definición de la asociación Pone: o sea, que no pueda haber más de una instancia de la relación Pone que conecten a un nodo Profesor y a un nodo Reclamacion.
- Cardinalidad máxima 1 de Reclamacion en la asociación Pone, es decir, un profesor puede presentar como mucho una reclamación.
- Cardinalidad mínima 1 del atributo motivo de la clase-asociación Asociada.
- Correcta definición de la clase-asociación Asociada: una instancia de esta clase-asociación se transforma en una relación Asociada entre un nodo Reclamacion y un

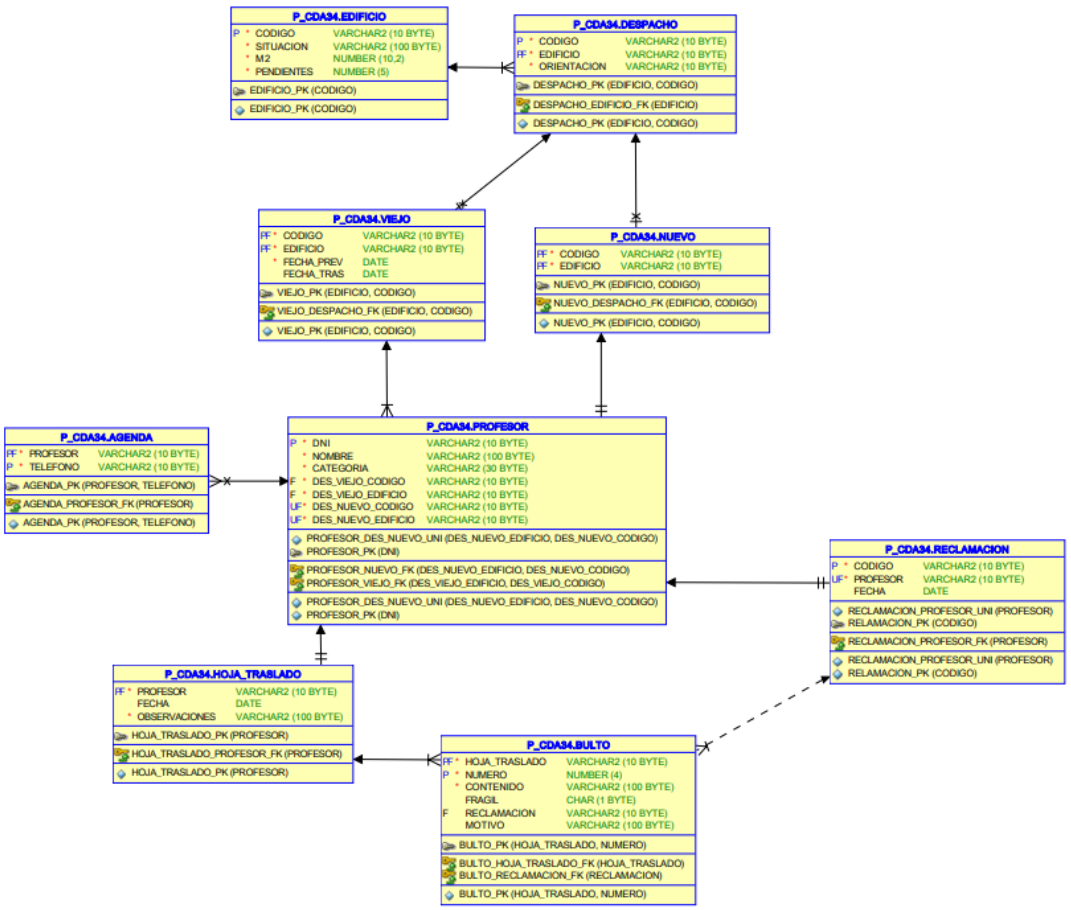
nodo Bulto. El atributo motivo de la clase-asociación se representa como propiedad de la relación Asociada.

- Coherencia de fechas: fecha_tras no debe ser anterior a fecha_prev.
- Los bultos asociados a una reclamación deben pertenecer a la hoja de traslado del profesor que reclama.

## 2.2 Tarea 4: Creación y carga de la base de datos Neo4j

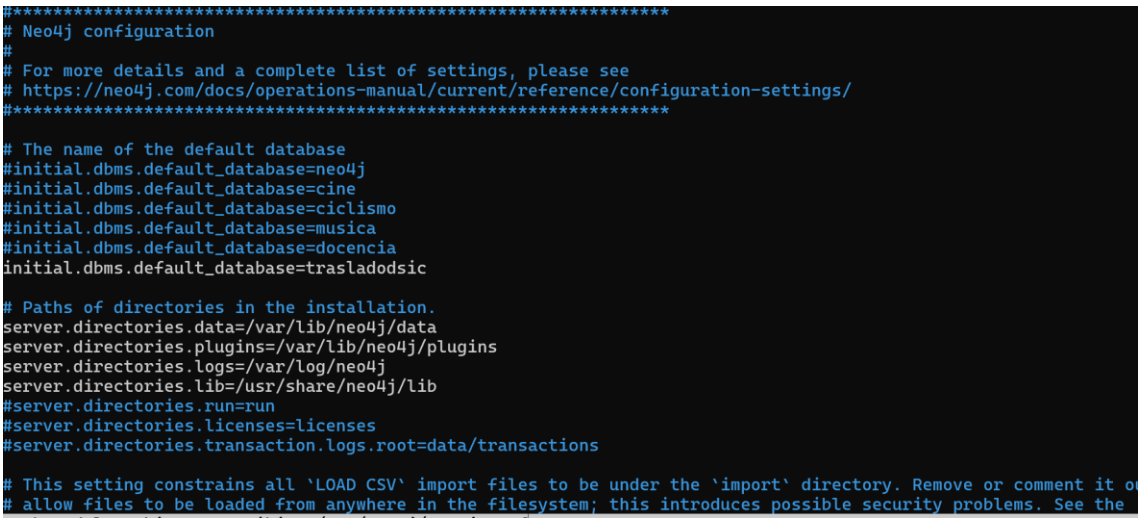
### 2.2.1 Diagrama de la base de datos Oracle generado con el Datamodeler

En la figura siguiente se muestra el esquema lógico de la base de datos ya creada en el Oracle.



## 2.2.2 Creación de la base de datos Traslado DSIC

Editamos el fichero *neo4j.config* y le añadimos una nueva base de datos a la que llamaremos *trasladodsic*:




## 2.2.3 Carga de los nodos

Se generan, a partir de la base de datos Oracle, ficheros csv para cargar después los nodos, propiedades y las relaciones en Neo4j, estos ficheros se suben al directorio /var/lib/neo4j/import/trasladodsic/ de la máquina virtual. A continuación se presentan las instrucciones *select* que permiten la generación de esos ficheros y las instrucciones *load* para realizar la carga.

### 2.2.3.1 Nodos :Edificio

- Consulta en Oracle para generar el fichero *edificio.csv*

```
SELECT código, cod_edif,  situacion,
     TO_CHAR(m2, '999999.99') m2, pendientes FROM
p_cda34.edificio;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///edificio.csv' AS x
CREATE (:Edificio {cod_edif: x.COD_EDIF,situacion: x.SITUACION,
m2: toFloat(x.M2), pendientes: toInteger(x.PENDIENTES)});
```

### 2.2.3.2 Nodos :Despacho:Nuevo

- Consulta en Oracle para generar el fichero *despachonuevo.csv*

```
SELECT n.edificio, n.codigo, d.orientacion
FROM p_cda34.nuevo n, p_cda34.despacho d
WHERE n.edificio= d.edificio AND n.codigo=d.codigo;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///despachonuevo.csv' AS x
CREATE (:Despacho:Nuevo { cod_edif: x.EDIFICIO,  cod_desp:
x.COdIGO, orientacion: x.ORIENTACION});
```

### 2.2.3.3 Nodos :Despacho:Viejo

- Consulta en Oracle para generar el fichero *despachoviejo.csv*

```
SELECT v.edificio,v.codigo,d.orientacion,
```

```
TO_CHAR(v.fecha_prev,'YYYY-MM-DD') fecha_prev,
TO_CHAR(v.fecha_tras,'YYYY-MM-DD') fecha_tras
FROM p_cda34.viejo v, p_cda34.despacho d
WHERE v.edificio= d.edificio AND v.codigo=d.codigo;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///despachoviejo.csv' AS x
CREATE (:Despacho:Viejo {cod_edif: x.EDIFICIO,cod_desp: x.CODIGO
,  orientacion: x.ORIENTACION,  fecha_prev: date(x.FECHA_PREV),
 fecha_tras: CASE x.FECHA_TRAS WHEN "" THEN nullELSE date(
x.FECHA_TRAS) END});
```

#### 2.2.3.4 Nodos :Profesor

- Consulta en Oracle para generar el fichero *profesor.csv*

```
SELECT
dni,nombre,categoria,des_viejo_codigo,des_viejo_edificio,des_nue
vo_codigo,des_nuevo_edificio
FROM p_cda34.profesor ;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///profesor.csv' AS x
CREATE (:Profesor {  dni: x.DNI,  nombre: x.NOMBRE,  categoria:
x.CATEGORIA,  telefono: []});
```

#### 2.2.3.5 Nodos :Telefono

- Consulta en Oracle para generar el fichero *telefono.csv*

```
SELECT profesor, telefono
FROM p_cda34.agenda;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///telefono.csv' AS x
MATCH (p:Profesor {dni: x.PROFESOR}) SET p.telefono =
coalesce(p.telefono, []) + x.TELEFONO;
```

#### 2.2.3.6 Nodos :Hoja_traslado

- Consulta en Oracle para generar el fichero *Hoja_traslado.csv*

```
SELECT profesor ,
TO_CHAR (fecha, 'YYYY-MM-DD') fecha_hoja,observaciones
FROM p_cda34.hoja_traslado ;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///Hoja_traslado.csv' AS x
CREATE (:Hoja_Traslado {profesor: x.PROFESOR,
fecha: date(x.FECHA), observaciones: x.OBSERVACIONES});
```

#### 2.2.3.7 Nodos :Bulto

- Consulta en Oracle para generar el fichero *bulto.csv*

```
SELECT hoja_traslado,numero,contenido,fragil
FROM p_cda34.bulto ;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///bulto.csv' AS x
```

```
CREATE (:Bulto {  hoja_traslado: x.HOJA_TRASLADO,  numero: toInt
eger(x.NUMERO), contenido: x.CONTENIDO,   fragil: CASE x.FRAGIL
WHEN "S" THEN true WHEN "C" THEN true WHEN "TRUE" THEN true
WHEN "True" THEN true WHEN "1" THEN true ELSE false  END});
```

#### 2.2.3.8 Nodos :Reclamación

- Consulta en Oracle para generar el fichero *reclamacion.csv*

```
SELECT codigo, profesor ,
TO_CHAR(fecha, 'YYYY-MM-DD') fecha_rec
FROM p_cda34.reclamacion;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///reclamacion.csv' AS x
CREATE (:Reclamacion {cod_rec: x.CODIGO,profesor: x.PROFESOR
,fecha: date(x.FECHA_REC)});
```

## 2.2.4 Carga de las relaciones entre nodos

#### 2.2.4.1 Relación Ubicado a partir del fichero *despachonuevo.csv*

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///despachonuevo.csv' AS x
MATCH (d:Despacho:Nuevo {cod_edif: x.EDIFICIO,  cod_desp:x.CODIG
O})MATCH (e:Edificio {  cod_edif:  x.EDIFICIO}) CREATE (d)-
[:Ubicado]->(e);
```

#### 2.2.4.2 Relación Ubicado a partir del fichero *despachoviejo.csv*

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///despachoviejo.csv' AS x
MATCH (d:Despacho:Viejo {cod_edif: x.EDIFICIO,cod_desp:
x.CODIGO}) MATCH (e:Edificio {cod_edif: x.EDIFICIO}) CREATE (d)-
[:Ubicado]->(e);
```

#### 2.2.4.3 Relación Ocupa_V a partir del fichero *profesor.csv*

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///profesor.csv' AS x
MATCH (p:Profesor {dni: x.DNI})MATCH  (d:Despacho:Viejo {  cod_e
dif: x.DES_VIEJO_EDIFICIO,  cod_desp: x.DES_VIEJO_CODIGO})CREATE
 (p)-[:Ocupa_V]->(d);
```

#### 2.2.4. Relación Ocupa_N a partir del fichero *profesor.csv*

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///profesor.csv' AS x
MATCH (p:Profesor {dni: x.DNI}) MATCH (d:Despacho:Nuevo {
cod_edif: x.DES_NUEVO_EDIFICIO,cod_desp: x.DES_NUEVO_CODIGO})CRE
ATE (p)-[:Ocupa_N]->(d);
```

#### 2.2.4.5 Relación Hace a partir del fichero *Hoja_traslado.csv*

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///Hoja_traslado.csv' AS x
MATCH (p:Profesor {dni:x.PROFESOR})MATCH (h:Hoja_Traslado {profe
sor: x.PROFESOR}) CREATE (p)-[:Hace]->(h);
```

#### 2.2.4.6 Relación Es a partir del fichero *bulto.csv*

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///bulto.csv' AS x
MATCH (h:Hoja_Traslado {profesor: x.HOJA_TRASLADO}) MATCH
(b:Bulto {hoja_traslado: x.HOJA_TRASLADO, numero:
toInteger(x.NUMERO)}) CREATE (h)-[:Es]->(b);
```

#### 2.2.4.7 Relación Pone a partir del fichero *reclamacion.csv*

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///reclamacion.csv' AS x
MATCH (p:Profesor {dni: x.PROFESOR})MATCH (r:Reclamacion {cod_re
c: x.CODIGO}) CREATE (p)-[:Pone]->(r);
```

#### 2.2.4.8 Relación Asociada a partir del fichero *bulto_reclamacion.csv*

- Consulta en Oracle para generar el fichero bulto_reclamacion.csv

```
SELECT cod_rec AS reclamacion,
       profesor AS hoja_traslado,
       numero AS numero,
       motivo AS motivo
FROM bulto_reclamado;
```

- Instrucción de carga en Neo4j

```
LOAD CSV WITH HEADERS FROM 'file:///bulto_reclamado.csv' AS x
MATCH (r:Reclamacion {cod_rec: x.RECLAMACION})
     MATCH (b:Bulto {hoja_traslado: x.HOJA_TRASLADO,numero: toIn
     tegerx.NUMERO)}) CREATE (r)-[:Asociada {motivo: x.MOTIVO}]-
     >(b);
```

### 2.2.5 Definición de las restricciones

- Unicidad sobre la propiedad *cod_edif* de los nodos *Edificio*

```
CREATE CONSTRAINT cod_edif_unico FOR (e:Edificio) REQUIRE
e.cod_edif IS UNIQUE;
```

- Unicidad sobre las propiedades *cod_edif y cod_desp* de los nodos *Viejo*

```
CREATE CONSTRAINT despacho_viejo_unico FOR (d:Viejo)
REQUIRE (d.cod_edif, d.cod_desp) IS UNIQUE;
```

- Unicidad sobre las propiedades *cod_edif y cod_desp* de los nodos *Nuevo*

```
CREATE CONSTRAINT despacho_nuevo_unico FOR (d:Nuevo)
REQUIRE (d.cod_edif, d.cod_desp) IS UNIQUE;
```

- Unicidad sobre la propiedad *dni* de los nodos *Profesor*

```
CREATE CONSTRAINT dni_profesor_unico
FOR (p:Profesor) REQUIRE p.dni IS UNIQUE;
```

- Unicidad sobre la propiedad *profesor* de los nodos *Hoja_traslado*

```
CREATE CONSTRAINT hoja_profesor_unico FOR (h:Hoja_Traslado)
REQUIRE h.profesor IS UNIQUE;
```

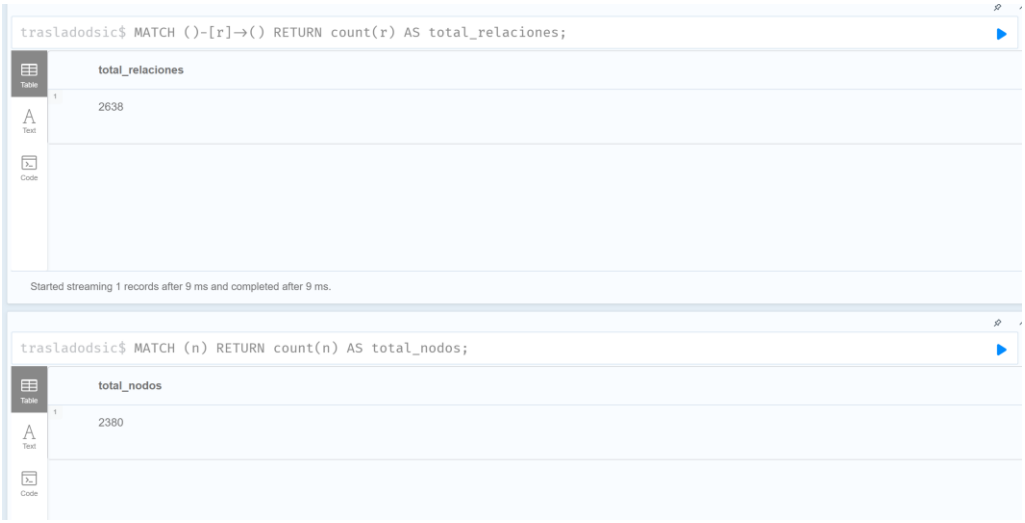
- Unicidad sobre las propiedades *hoja_traslado y numero* de los nodos *Bulto*

```
CREATE CONSTRAINT bulto_unico FOR (b:Bulto) REQUIRE
(b.hoja_traslado, b.numero) IS UNIQUE;
```

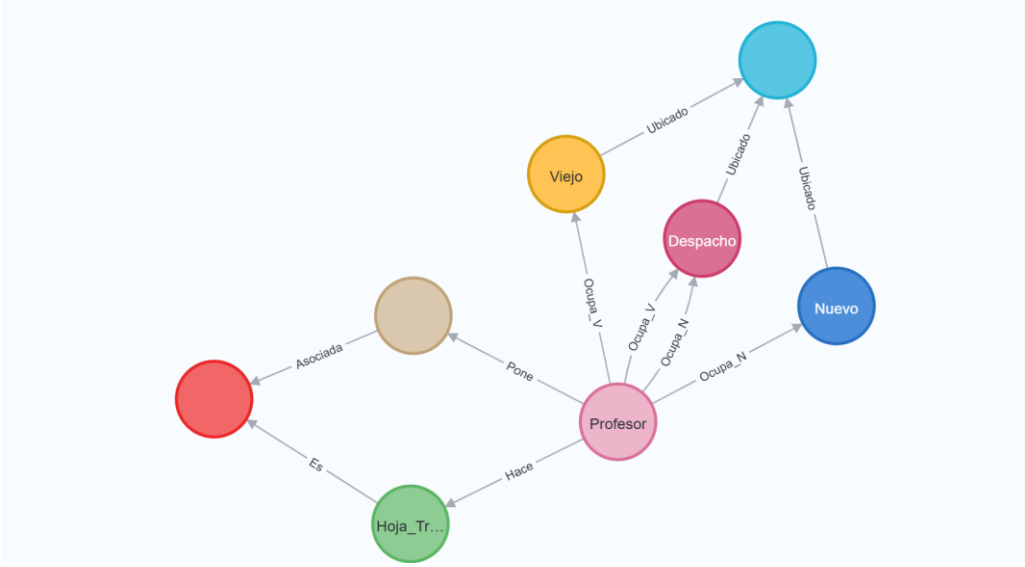
- Unicidad sobre la propiedad *cod_rec* de los nodos *Reclamacion*

```
CREATE CONSTRAINT cod_rec_unico FOR (r:Reclamacion) REQUIRE
r.cod_rec IS UNIQUE;
```

## 2.2.6 Esquema Neo4j




Como se puede observar hay 2380 nodos y 2638 relaciones



## 2.3 Tarea 5: Consultas en Cypher

1) Listar todos los edificios con sus datos.

```
MATCH (e:Edificio) RETURN e.cod_edif AS edificio, e.situacion AS
situacion, e.m2 AS metros_cuadrados, e.pendientes AS
despachos_pendientes
ORDER BY e.cod_edif;
```

| | edificio | situacion | metros_cuadrados | despachos_pendientes |
|---|---|---|---|---|
| 1 | "E01" | "Situacion_1" | 1574.49 | 0 |
| 2 | "E02" | "Situacion_2" | 644.95 | 0 |
| 3 | "E03" | "Situacion_3" | 928.77 | 0 |
| 4 | "E04" | "Situacion_4" | 618.11 | 0 |
| 5 | "E05" | "Situacion_5" | 540.06 | 0 |
| 6 | "E06" | "Situacion_6" | 576.44 | 0 |
| 7 | | | | |

2) Contar cuántos despachos hay en cada edificio.

```
MATCH (d:Despacho)-[:Ubicado]->(e:Edificio)
RETURN e.cod_edif AS edificio, count(d) AS total_despachos
ORDER BY e.cod_edif;
```

| | edificio | total_despachos |
|---|---|---|
| 1 | "E08" | 500 |

3) Consultar el despacho viejo y el despacho nuevo de cada profesor.

```
MATCH (p:Profesor)-[:Ocupa_V]->(v:Despacho:Viejo), (p)-
[:Ocupa_N] >(n:Despacho:Nuevo)
RETURN p.dni AS dni, p.nombre AS nombre,p.categoria AS
categoria, v.cod_edif AS edificio_viejo, v.cod_desp AS
despacho_viejo, n.cod_edif AS edificio_nuevo, n.cod_desp AS
despacho_nuevo
ORDER BY p.nombre;
```

| | dni | nombre | categoria | edificio_viejo | despacho_viejo | edificio_nuevo | despacho_nuevo |
|---|---|---|---|---|---|---|---|
| 1 | "P0001" | "Profesor_1" | "ASOCIADO" | "E08" | "D0249" | "E08" | "D0364" |
| 2 | "P0010" | "Profesor_10" | "ASOCIADO" | "E08" | "D0009" | "E08" | "D0190" |
| 3 | "P0100" | "Profesor_100" | "ASOCIADO" | "E08" | "D0020" | "E08" | "D0276" |
| 4 | "P0101" | "Profesor_101" | "AYUDANTE" | "E08" | "D0037" | "E08" | "D0091" |
| 5 | "P0102" | "Profesor_102" | "TITULAR" | "E08" | "D0325" | "E08" | "D0458" |
| 6 | "P0103" | "Profesor_103" | "ASOCIADO" | "E08" | "D0299" | "E08" | "D0429" |
| 7 | | | | | | | |

4) Listar los profesores que tienen hoja de traslado.

```
MATCH (p:Profesor)-[:Hace]->(h:Hoja_Traslado)
RETURN p.dni AS dni, p.nombre AS nombre, h.fecha AS fecha_hoja,
h.observaciones AS observaciones
ORDER BY h.fecha;
```

| | dni | nombre | fecha_hoja | observaciones |
|---|---|---|---|---|
| 1 | "P0010" | "Profesor_10" | *null* | "Traslado_P0010" |
| 2 | "P0160" | "Profesor_160" | *null* | "Traslado_P0160" |
| 3 | "P0027" | "Profesor_27" | *null* | "Traslado_P0027" |
| 4 | "P0044" | "Profesor_44" | *null* | "Traslado_P0044" |
| 5 | "P0047" | "Profesor_47" | *null* | "Traslado_P0047" |
| 6 | "P0007" | "Profesor_7" | *null* | "Traslado_P0007" |
| 7 | | | | |

5) Listar los bultos de cada profesor, indicando si son frágiles.

```
MATCH (p:Profesor)-[:Hace]->(h:Hoja_Traslado)-[:Es]->(b:Bulto)
RETURN p.dni AS dni, p.nombre AS nombre, b.numero AS
numero_bulto,
b.contenido AS contenido, b.fragil AS fragil
ORDER BY p.dni, b.numero;
```

| | dni | nombre | numero_bulto | contenido | fragil |
|---|---|---|---|---|---|
| 1 | "P0001" | "Profesor_1" | 1 | "Contenido_1" | false |
| 2 | "P0001" | "Profesor_1" | 2 | "Contenido_2" | true |
| 3 | "P0001" | "Profesor_1" | 3 | "Contenido_3" | false |
| 4 | "P0001" | "Profesor_1" | 4 | "Contenido_4" | true |
| 5 | "P0001" | "Profesor_1" | 5 | "Contenido_5" | false |
| 6 | "P0001" | "Profesor_1" | 6 | "Contenido_6" | false |

6) Consultar qué profesores han presentado reclamación y los datos de la reclamación.

```
MATCH (p:Profesor)-[:Pone]->(r:Reclamacion)
RETURN p.dni AS dni, p.nombre AS nombre, r.cod_rec AS
reclamacion, r.fecha AS fecha_reclamacion
ORDER BY r.fecha;
```

| | dni | nombre | reclamacion | fecha_reclamacion |
|---|---|---|---|---|
| 1 | "P0014" | "Profesor_14" | "R020" | "2024-01-04" |
| 2 | "P0013" | "Profesor_13" | "R004" | "2024-01-13" |
| 3 | "P0192" | "Profesor_192" | "R010" | "2024-01-24" |
| 4 | "P0050" | "Profesor_50" | "R018" | "2024-01-25" |
| 5 | "P0186" | "Profesor_186" | "R005" | "2024-01-28" |
| 6 | "P0149" | "Profesor_149" | "R009" | "2024-01-29" |
| 7 | | | | |

7) Obtener los profesores que tienen más de 5 bultos en su hoja de traslado.

```
MATCH (p:Profesor)-[:Hace]->(h:Hoja_Traslado)-[:Es]->(b:Bulto)
WITH p, count(b) AS num_bultos
WHERE num_bultos > 5
RETURN p.dni AS dni, p.nombre AS nombre, p.categoria AS
categoria, num_bultos AS numero_bultos
ORDER BY num_bultos DESC, p.nombre;
```

| dni | nombre | categoria | numero_bultos |
|---|---|---|---|
| "P0001" | "Profesor_1" | "ASOCIADO" | 10 |
| "P0010" | "Profesor_10" | "ASOCIADO" | 10 |
| "P0101" | "Profesor_101" | "AYUDANTE" | 10 |
| "P0103" | "Profesor_103" | "ASOCIADO" | 10 |
| "P0104" | "Profesor_104" | "AYUDANTE" | 10 |
| "P0106" | "Profesor_106" | "ASOCIADO" | 10 |

8) Contar cuántos bultos reclamados tiene cada reclamación.

```
MATCH (p:Profesor)-[:Pone]->(r:Reclamacion)-[:Asociada]-
>(b:Bulto)
RETURN p.dni AS dni, p.nombre AS nombre, r.cod_rec AS
reclamacion, count(b) AS bultos_reclamados
ORDER BY bultos_reclamados DESC, r.cod_rec;
```

| | dni | nombre | reclamacion | bultos_reclamados |
|---|---|---|---|---|
| 1 | "P0086" | "Profesor_86" | "R001" | 5 |
| 2 | "P0079" | "Profesor_79" | "R006" | 5 |
| 3 | "P0149" | "Profesor_149" | "R009" | 5 |
| 4 | "P0173" | "Profesor_173" | "R014" | 5 |
| 5 | "P0112" | "Profesor_112" | "R016" | 5 |
| 6 | "P0014" | "Profesor_14" | "R020" | 5 |
| 7 | | | | |